Computers & Security 139 (2024) 103676

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

EvadeDroid: A practical evasion attack on machine learning for black-box Android malware detection

Hamid Bostani [a,*], Veelasha Moonsamy [b]

[a] Digital Security Group, Institute for Computing and Information Sciences, Radboud University, Nijmegen, the Netherlands
[b] Horst Görtz Institute for IT Security, Ruhr University Bochum, Bochum, Germany

ARTICLE INFO

Keywords:
Query-based evasion attacks
Android malware detection
Machine learning
Black-box adversarial attacks

ABSTRACT

Over the last decade, researchers have extensively explored the vulnerabilities of Android malware detectors to adversarial examples through the development of evasion attacks; however, the practicality of these attacks in real-world scenarios remains arguable. The majority of studies have assumed attackers know the details of the target classifiers used for malware detection, while in reality, malicious actors have limited access to the target classifiers. This paper introduces *EvadeDroid*, a problem-space adversarial attack designed to effectively evade black-box Android malware detectors in real-world scenarios. EvadeDroid constructs a collection of problem-space transformations derived from benign donors that share opcode-level similarity with malware apps by leveraging an *n*-gram-based approach. These transformations are then used to morph malware instances into benign ones via an iterative and incremental manipulation strategy. The proposed manipulation technique is a query-efficient optimization algorithm that can find and inject optimal sequences of transformations into malware apps. Our empirical evaluations, carried out on $1K$ malware apps, demonstrate the effectiveness of our approach in generating real-world adversarial examples in both soft- and hard-label settings. Our findings reveal that EvadeDroid can effectively deceive diverse malware detectors that utilize different features with various feature types. Specifically, EvadeDroid achieves evasion rates of 80%–95% against DREBIN, Sec-SVM, ADE-MA, MaMaDroid, and Opcode-SVM with only 1–9 queries. Furthermore, we show that the proposed problem-space adversarial attack is able to preserve its stealthiness against five popular commercial antiviruses with an average of 79% evasion rate, thus demonstrating its feasibility in the real world.

1. Introduction

Machine Learning (ML) continues to show promise in detecting sophisticated and zero-day malicious programs (Abri et al., 2019; Ahmed et al., 2009; Bai et al., 2014; Eskandari et al., 2013; Firdausi et al., 2010; Raff and Nicholas, 2017; Venkatraman et al., 2019). However, despite the effectiveness of ML-based malware detectors, these defense strategies are vulnerable to *evasion attacks* (Li et al., 2021a). More concretely, attackers aim to deceive ML-based malware classifiers by transforming existing malware into *adversarial examples* (AEs) via a series of manipulations. The proliferation of Android malware (Castillo and Samani, 2021) has extended research into novel evasion attacks to strengthen malware classifiers against AEs (Berger et al., 2020; Cara et al., 2020; Chen et al., 2017, 2018, 2019; Demontis et al., 2017; Grosse et al., 2017; Li et al., 2020; Liu et al., 2019; Pierazzi et al., 2020; Rathore et al., 2021; Xu et al., 2021; Yang et al., 2017). However, this endeavor, which also exists for other platforms, such as Windows, poses its own set of challenges, which we elaborate on further below.

The first challenge pertains to the *feature representation* of Android applications (apps). Making a slight modification in the feature representation of a malware app may break its functionality (Li et al., 2021a) as malware features extracted from Android Application Packages (APKs) are usually discrete (e.g., app permissions) instead of continuous (e.g., pixel intensity in a grayscale image). One plausible solution is to manipulate the features extracted from the Android Manifest file (Berger et al., 2020; Grosse et al., 2017; Rathore et al., 2021); however, the practicality of such manipulations in generating executable AEs is questionable for the following reasons. Firstly, modifying features from the Android Manifest (e.g., content providers, intents, etc.) cannot guarantee the executability of the original apps (i.e., malicious payload) (Khormali et al., 2019; Li et al., 2020). Secondly, adding unused features to the Manifest file can be discarded by applying pre-processing

* Corresponding author.
E-mail addresses: hamid.bostani@ru.nl (H. Bostani), email@veelasha.org (V. Moonsamy).

https://doi.org/10.1016/j.cose.2023.103676
Received 21 September 2023; Received in revised form 4 December 2023; Accepted 20 December 2023
Available online 27 December 2023
0167-4048/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

techniques (Pierazzi et al., 2020). Finally, advanced Android malware detectors (e.g., Ma et al., 2019; Onwuzurike et al., 2019) primarily rely on the semantics of Android apps, which are represented by the Dalvik bytecode rather than the Manifest files (Berger et al., 2020; Chen et al., 2019).

Another challenge is the limitations of *feature mapping* techniques used to convert Android apps from the problem space (i.e., input space) to feature space. These techniques are not reversible, meaning that feature-space perturbations cannot be directly translated into a malicious app (Pierazzi et al., 2020). To address *inverse feature-mapping problem*, a common approach is to manipulate real-world malware apps using problem-space transformations that correspond to the features used in ML models. By applying these feature-based transformations to Android apps, adversaries can create hazardous evasion attacks (Cara et al., 2020; Chen et al., 2019; Pierazzi et al., 2020; Yang et al., 2017). However, finding suitable transformations that satisfy problem-space constraints is not straightforward (Pierazzi et al., 2020): Firstly, certain transformations (e.g., Li and Li, 2020; Li et al., 2021b) intended to mimic feature-space perturbations may not result in feasible AEs because they disregard feature dependencies from real-world objects. Additionally, some transformations (e.g., Cara et al., 2020; Pierazzi et al., 2020) that meet problem-space constraints for manipulating real objects may introduce undesired or incompatible payloads into malware apps. These types of transformations not only might render the perturbations different from what the attacker expects (Cara et al., 2020) but can also lead to the crashing of adversarial malware apps.

The final challenge revolves around current methods (Berger et al., 2020; Cara et al., 2020; Chen et al., 2017, 2018, 2019; Demontis et al., 2017; Grosse et al., 2017; Li and Li, 2020; Li et al., 2021b; Liu et al., 2019; Pierazzi et al., 2020; Rathore et al., 2021; Xu et al., 2021) that generate AEs based on the specifics of target malware detectors, such as the ML algorithm and feature set. These approaches assume that attackers possess either *Perfect Knowledge* (PK) or *Limited Knowledge* (LK) about the target classifiers. However, in real-world scenarios, adversaries generally have *Zero Knowledge* (ZK) about the target malware detectors, which aligns more closely with reality since antivirus systems operate as black-box engines that are queried (Rosenberg et al., 2020). Some studies (Croce et al., 2022; Xu et al., 2021; Zhang et al., 2021) have explored semi-black-box settings to generate AEs by leveraging feedback from the target detectors. Nevertheless, these approaches suffer from inefficiency in terms of evasion costs, including the high number of queries required and the extent of manipulation applied to the input sample. Efficient querying is crucial due to the associated costs (Rosenberg et al., 2020) and the risk of detectors blocking suspicious queries. Additionally, minimizing manipulation is desired as excessive manipulations could impact the malicious functionality of apps (Demontis et al., 2017).

1.1. Contributions

In response to the challenges outlined earlier, we propose a comprehensive and generalized evasion attack called *EvadeDroid*, which can bypass black-box Android malware classifiers through a two-step process: (i) *preparation* and (ii) *manipulation*. The first step involves implementing a donor selection technique within EvadeDroid to create an action set comprising a collection of problem-space transformations, i.e., code snippets known as *gadgets*. These gadgets are derived by performing program slicing on benign apps (i.e., donors) that are publicly available. By injecting each gadget into a malware app, specific payloads from a benign donor can be incorporated into the malware app. Our proposed technique utilizes an *n-gram-based similarity* method to identify suitable donors, particularly benign apps that exhibit similarities to malware apps at the opcode level. Applying transformations derived from these donors to malware apps can enable them to appear benign or move them towards blind spots of ML classifiers. This approach aims to achieve the desired outcome of introducing transformations that not only ensure adherence to problem-space constraints (i.e., preserved semantics, robustness to preprocessing, and plausibility (Pierazzi et al., 2020)) but also lead to malware classification errors.

In the manipulation step, EvadeDroid uses an iterative and incremental manipulation strategy to create real-world AEs. This procedure incrementally perturbs malware apps by applying a sequence of transformations gathered in the action set into malware samples over several iterations. We propose a search method to randomly choose suitable transformations and apply them to malware apps. The random search algorithm, which moves malware apps in the problem space, is guided by the labels of manipulated malware apps. These labels are specified by querying the target black-box ML classifier. Our contributions can be summarized as follows:

- We propose a *black-box evasion* attack that generates real-world Android AEs that adhere to problem-space constraints. To the best of our knowledge, EvadeDroid is the pioneer study in the Android domain that successfully evades ML-based malware detectors by directly manipulating malware samples without performing feature-space perturbations.
- We demonstrate that EvadeDroid is a *query-efficient* attack capable of deceiving various black-box ML-based malware detectors through minimal querying. Specifically, our proposed problem-space adversarial attack achieves evasion rates of 89%, 85%, 86%, 95%, and 80% against DREBIN (Arp et al., 2014), Sec-SVM (Demontis et al., 2017), ADE-MA (Li and Li, 2020), MaMaDroid (Onwuzurike et al., 2019), and Opcode-SVM (Jerome et al., 2014), respectively. This research represents one of the pioneering efforts in the Android domain, introducing a realistic problem-space attack in a ZK setting.
- Our proposed attack can operate with either *soft labels* (i.e., confidence scores) or *hard labels* (i.e., classification labels) of malware apps, as specified by the target malware classifiers, to generate AEs.
- We assess the practicality of the proposed evasion attack under real-world constraints by evaluating its performance in deceiving popular commercial antivirus products. Specifically, our findings indicate that EvadeDroid can significantly diminish the effectiveness of five popular commercial antivirus products, achieving an average evasion rate of approximately 79%.
- In the spirit of open science and to allow reproducibility, we have made our code available at https://github.com/HamidBostani2021/EvadeDroid

The rest of the paper is organized as follows: §2 reviews the most important relevant studies, particularly in the Android domain. In §3, we provide background information on fundamental concepts, specifically ML-based malware detectors, and briefly discuss the practical transformations that can be used for manipulating APKs. §4 initiates by reviewing the threat model and articulating the problem definition for EvadeDroid. Following this, an illustration of the proposed black-box attack will be presented. We evaluate EvadeDroid's performance in §5. Limitations and future work, along with a brief conclusion, are presented in §6 and §7.

2. Related work

In the past few years, several studies have explored AEs in the context of malware, particularly in the Windows domain. For example, Demetrio et al. (2021) generated AEs in a black-box setting by applying structural and behavioral manipulations. Song et al. (2022) employed code randomization techniques to generate real-world AEs. They proposed an adversarial framework guided by reinforcement learning to model the action selection problem as a multi-armed bandit problem. Sharif et al. (2019) used binary diversification techniques to evade malware detection. Khormali et al. (2019) bypassed visualization-based malware detectors by applying padding and sample injection to mal-

Table 1
Evasion attacks in ML-based Android Malware Detectors.

Relevant Papers	Attacker's Knowledge			Perturbation Type	
	PK	LK	ZK	Problem Space	Feature Space
Xu et al. (2023)			✓	✓	✓
He et al. (2023)			✓	✓	✓
Li et al. (2023)		✓		✓	✓
Croce et al. (2022)		✓			✓
Zhang et al. (2021)		✓		✓	✓
Rathore et al. (2021)	✓	✓			✓
Chen et al. (2017)	✓	✓			✓
Demontis et al. (2017)	✓	✓	✓	✓	✓
Grosse et al. (2017)	✓			✓	✓
Chen et al. (2018)	✓	✓			✓
Liu et al. (2019)		✓			✓
Xu et al. (2021)		✓			✓
Berger et al. (2020)	✓	✓		✓	✓
Pierazzi et al. (2020)	✓			✓	✓
Chen et al. (2019)		✓		✓	✓
Cara et al. (2020)		✓		✓	✓
Yang et al. (2017)			✓	✓	✓
Li et al. (2021b)	✓	✓		✓	✓
Li and Li (2020)	✓	✓		✓	✓
EvadeDroid			✓	✓	

ware samples. Demetrio et al. (2019) generated adversarial malware by making small manipulations in the file headers of malware samples. Rosenberg et al. (2020) presented a black-box attack that perturbs API sequences of malware samples to mislead malware classifiers.

While evasion attacks have made significant advancements in the Windows domain, their effectiveness in the Android domain may be limited because their manipulations might not be appropriate for altering Android malware apps in a way that can deceive existing Android malware detectors. Over the last few years, various studies have been performed to generate AEs in the Android ecosystem to anticipate possible evasion attacks. Table 1 illustrates the threat models that were considered by researchers. Note that in the categorization of studies under the ZK setting, adversaries should not only lack access to the details of the target model but also have no assumptions (e.g., types of features utilized by detectors) about it. To study feature-space AEs, Croce et al. (2022) introduced Sparse-RS, a query-based attack that generated AEs using a random search strategy. Rathore et al. (2021) generated AEs by using Reinforcement Learning to mislead Android malware detectors. Chen et al. (2017, 2018) implemented different feature-based attacks (e.g., brute-force attacks) to evaluate their defense strategies. Demontis et al. (2017) presented a white-box attack to perturb feature vectors of Android malware apps regarding the most important features that impact the malware classification. Liu et al. (2019) introduced an automated testing framework based on a Genetic Algorithm (GA) to strengthen ML-based malware detectors. Xu et al. (2021) proposed a semi-black-box attack that perturbs features of Android apps based on the simulated annealing algorithm. The above attacks seem impractical as they do not show how real-world apps can be reconstructed based on feature-space perturbations.

To investigate problem-space manipulations, Grosse et al. (2017) manipulated the Android Manifest files based on the feature-space perturbations. Berger et al. (2020) and Li and Li (2020); Li et al. (2021b) used a similar approach; however, they considered both Manifest files and Dalvik bytecodes of Android apps in their modification methods. Zhang et al. (2021) introduced an adversarial attack called *Shadow-Droid* to generate AEs using a substitute model built on permissions and API call features. Xu et al. (2023) introduced GenDroid, a query-based attack that employed GA by integrating an evolutionary strategy based on Gaussian Process Regression. The practicality of these attacks is also questionable because the generated AEs might not satisfy all the constraints in the problem space (Pierazzi et al., 2020) (e.g., plausibility and robustness to preprocessing). For instance, Li et al. (2021b) reported that 5 out of 10 manipulated apps that were validated could not run successfully. Furthermore, unused features injected into APKs by the attacks discussed in Berger et al. (2020); Grosse et al. (2017); Li and Li (2020); Li et al. (2021b); Xu et al. (2023); Zhang et al. (2021) not only raise plausibility concerns but also render them susceptible to elimination by preprocessing operator (Pierazzi et al., 2020), especially those features incorporated into Manifest files.

In addition to the aforementioned studies, some (e.g., Cara et al., 2020; Chen et al., 2019; Pierazzi et al., 2020; Yang et al., 2017) have considered the *inverse feature-mapping problem* when presenting practical AEs in the Android domain. Pierazzi et al. (2020) proposed a problem-space adversarial attack to generate real-world AEs by applying functionality-preserving transformations to the input malware apps. Chen et al. (2019) added adversarial perturbations found by a substitute ML model to Android malware apps. Cara et al. (2020) presented a practical evasion attack by injecting system API calls determined via mimicry attack on APKs. Li et al. (2023) proposed a problem-space attack called *BagAmmo*, targeting function call graph (FCG) based malware detection. The main shortcoming of these studies is that the authors assume the adversary to have perfect knowledge (Pierazzi et al., 2020) or limited knowledge (Cara et al., 2020; Chen et al., 2019) about the target classifiers (e.g., knowing the feature space or accessing the training set), while in real scenarios (e.g., bypassing antivirus engines), an adversary often has zero knowledge about the target malware detectors. For instance, BagAmmo (Li et al., 2023) assumes that the target malware detector is based on FCG, which implies that it has some knowledge about the target model. Note that this assumption may not be applicable in all real-world scenarios, as different malware detectors may employ diverse feature sets.

On the other hand, despite the practicality of Pierazzi et al. (2020) in attacking white-box malware classifiers, the side-effect features that appear from undesired payloads injected into malware samples may manipulate the feature representations of apps differently from what the attacker expects (Cara et al., 2020). Furthermore, such attacks may cause the adversarial malware to grow infinitely in size as they do not consider the size constraint of the adversarial manipulations. The attacks presented in Chen et al. (2019); Li et al. (2023) are tailored to the target malware classifiers (i.e., DREBIN (Arp et al., 2014), and FCG-based detectors such as MaMaDroid (Onwuzurike et al., 2019)), which means the authors did not succeed in presenting a generalized evasion technique. Moreover, the attack in Cara et al. (2020) has some limitations, such as injecting incompatible APIs into Android apps or using incorrect parameters for API calls, which can crash adversarial malware apps.

To address the aforementioned shortcomings, Yang et al. (2017) proposed two attacks named the *evolution* and *confusion* attacks, designed to evade target classifiers in a black-box setting. However, their approach lacks details about critical issues (e.g., the feature extraction method) and is impractical because, as reported by the authors, their attacks can easily disrupt the functionality of APKs after a few manipulations. Demontis et al. (2017) employed an obfuscation tool to bypass Android malware classifiers, but their results indicate a low performance for their method. He et al. (2023) introduced a query-based attack utilizing a perturbation selection tree and an adjustment policy. Nevertheless, the proposed attack is ineffective in hard-label settings, which are crucial for most real-world scenarios. Furthermore, in addition to the questionable plausibility of this attack, its success would be jeopardized by the disputable assumption that perturbations in the attack's malware perturbation set impact the feature values of target malware detectors.

EvadeDroid addresses the limitations of existing attacks by thoroughly aiming to meet the practical demands of real-world scenarios, such as hard-label attacking in a fully ZK setting, query efficiency, and satisfaction of all problem-space constraints. The novelty of our work, compared to the aforementioned studies, lies in the following aspects: (i) EvadeDroid provides adversaries with a general tool to bypass various Android malware detectors, as it is a problem-space evasion attack that operates in a ZK setting without any pre-assumptions about the features and types of features employed by the target malware detectors (§5.2). (ii) Unlike other evasion attacks, EvadeDroid directly manipulates Android apps without relying on feature-space perturbations. Its transformations not only are independent of the feature space but also adhere to problem-space constraints (§4.1). (iii) EvadeDroid is simple and easy to implement in real-world scenarios (§5.4) with proper transferability (§5.5). It is a query-efficient evasion attack that only requires the hard labels of Android apps provided by target black-box malware detectors (e.g., cloud-based antivirus services) (§5.2).

3. Background

In this section, we present a concise overview of the fundamental backgrounds relevant to Android evasion attacks. This encompasses the structure of Android apps, ML-based Android malware detection, and the adversarial transformations used for generating Android adversarial malware.

3.1. Android application package (APK)

APK is a compressed file format with a *.apk* extension. APKs contain various contents such as Resources and Assets. However, the most crucial contents, particularly for malware detectors, are the Manifest (AndroidManifest.xml) and Dalvik bytecode (classes.dex). The Manifest is an XML file that provides essential information about Android apps, including the package name, permissions, and definitions of Android components. It contains all the metadata required by the Android OS to install and run Android apps. On the other hand, Dalvik bytecode, also known as Dalvik Executable or DEX file, is an executable file that represents the behavior of Android apps.

Apktool (Apktool, 2010) is a popular reverse-engineering tool for the static analysis of Android apps. This reverse-engineering instrument can decompile and recompile Android apps. In the decompilation process, the DEX files of Android apps are compiled into a human-readable code called *smali*. Besides the above tool, *Soot* (Vallée-Rai et al., 1999) and *FlowDroid* (Arzt et al., 2014) are two Java-based frameworks that are used for analyzing Android apps. Soot extracts different information from APKs (e.g., API calls) which are then used during static analysis. One of the advantages of Soot for malware detection is its ability to generate call graphs; however, Soot cannot generate accurate call graphs for all apps because of the complexity of the control flow of some APKs. To address this shortcoming, FlowDroid, which is a Soot-based framework, can create precise call graphs based on the app's life cycle. It is worth noting that EvadeDroid uses Apktool, FlowDroid, and Soot in different components of its pipeline to generate adversarial examples.

3.2. ML-based Android malware detection

Leveraging ML for malware detection has garnered significant interest among cybersecurity researchers in the past decade. ML has demonstrated its potential as an effective solution in static malware analysis, enabling the identification of sophisticated and previously unknown malware through the generalization capabilities of ML algorithms (Li et al., 2021a). It is important to note that static analysis is a prominent approach for detecting malicious programs, where apps are classified based on their source code (i.e., static features) without execution. This approach offers fast analysis, allowing for the examination of an app's code comprehensively, with minimal resource usage in terms of memory and CPU (Jusoh et al., 2021). In order to represent programs for ML algorithms, various types of features are commonly employed in the static analysis, including syntax features (e.g., requested permissions and API calls (Arp et al., 2014; Demontis et al., 2017; Li and Li, 2020)), opcode features (e.g., n-gram opcodes (Kang et al., 2016)), image features (e.g., grayscale representations of bytecodes (Han et al., 2015)), and semantic features (e.g., function call graphs (Onwuzurike et al., 2019)).

3.3. Adversarial transformations

In the programming domain, a safe transformation refers to a problem-space transformation that maintains the semantic equivalence of the original program while ensuring its executability. In the adversarial malware domain, safe transformations, which guarantee preserved-semantics constraint, can become adversarial transformations if they are also plausible and robust to processing (refer to Appendix A for additional details regarding these constraints). Generally, in the context of Android malware detection, attackers have three types of adversarial transformations at their disposal to manipulate malicious apps (Pierazzi et al., 2020): (i) *feature addition*, (ii) *feature removal*, and (iii) *feature modification*. Feature addition involves adding new elements, such as API calls, to the programs, while feature removal entails removing contents like user permissions. Feature modification combines both addition and removal transformations in malware programs. Most studies have primarily focused on feature addition, as removing features from the source code is a complex operation that may cause malware apps to crash. Code transplantation (Pierazzi et al., 2020; Yang et al., 2017), system-predefined transformation (Cara et al., 2020), and dummy transformation (Berger et al., 2020; Chen et al., 2019; Grosse et al., 2017; Li and Li, 2020; Li et al., 2021b) are three potential methods for adding features to manipulate Android apps. However, two main issues arise when considering feature additions:

(i) What specific content should be included. By deriving problem-space transformations from feature-space perturbations, the attacker aims to ensure that the additional contents (e.g., API calls, Activities, etc.) are guaranteed to appear in the feature vector of the manipulated malware app (Pierazzi et al., 2020). Therefore, attackers may either use dummy contents (e.g., functions, classes, etc.) (Chen et al., 2019) or system-predefined contents (e.g., Android system packages) (Cara et al., 2020) for this purpose. As the plausibility of these transformations is debatable due to the potential lack of complete inconspicuousness, malicious actors may also make use of content present in already-existing Android apps. The *automated software transplantation* technique (Barr et al., 2015) can then be used to allow attackers to successfully carry out safe transformations. They extract some slices of existing bytecodes from benign apps (i.e., donor) during the *organ harvesting* phase, and the collected payloads are injected into malware apps in the *organ transplantation* phase.

(ii) Where contents should be injected. New contents must preserve the semantics of malware samples; therefore, they should be injected into areas that cannot be executed during runtime. For example, new contents can be added after `RETURN` instructions (Demontis et al., 2017) or inside an `IF` statement that is always false (Pierazzi et al., 2020). However, these injected contents are not robust to preprocessing if static analysis can discard unreachable code. One creative idea to add unreachable code that is undetectable is the use of *opaque predicates* (Moser et al., 2007). In this approach, new contents are injected inside an `IF` statement where its outcome can only be determined at runtime (Pierazzi et al., 2020).

4. Proposed attack

Here we first review the threat model and the problem definition of EvadeDroid. Subsequently, we will offer an illustration of the proposed attack.

4.1. Threat model

Adversarial Goal. The purpose of EvadeDroid is to manipulate Android malware samples in order to deceive static ML-based Android malware detectors. The proposed attack is an untargeted attack (Carlini et al., 2019) designed to mislead binary classifiers utilized in Android malware detection, causing Android malware apps to be misclassified. In other words, EvadeDroid's objective is to trick malware classifiers into classifying malware samples as benign.

Adversarial Knowledge. The proposed evasion attack has black-box access to the target malware classifier. Therefore, EvadeDroid does not have knowledge of the training data D, the feature set X, or the classification model f (i.e., the classification algorithm and its hyper-parameters). The attacker can only obtain the classification results (e.g., hard labels or soft labels) by querying the target malware classifier.

Adversarial Capabilities. EvadeDroid is designed to deceive black-box Android malware classifiers during their prediction phase. Our attack manipulates an Android malware app by applying a set of safe transformations, known as Android gadgets (i.e., slices of the benign apps' bytecode), which are optimized through interactions with the black-box target classifier. To ensure adherence to problem-space constraints, EvadeDroid leverages a tool, developed by the authors (Pierazzi et al., 2020), for extracting and injecting gadgets. Furthermore, in order to avoid major disruptions to apps, the manipulation process of a malware app is conducted gradually, making it resemble benign apps. This is achieved by injecting a minimal number of gadgets extracted from benign apps into the malware app, and the process continues until the malware app is misclassified or reaches the predefined evasion cost. In addition to the problem-space constraints discussed in previous research (Pierazzi et al., 2020), EvadeDroid must also adhere to two additional constraints highlighting the significance of minimizing evasion costs:

- **Number of queries.** EvadeDroid is a decision-based adversarial attack that aims to generate AEs while minimizing the number of queries, thus reducing the associated costs (Rosenberg et al., 2020).
- **Size of adversarial payloads.** In order to generate executable and visually inconspicuous AEs, such as those with minimal file size (Demetrio et al., 2021), EvadeDroid aims to minimize the size of injected adversarial payloads.

It is worth mentioning, each gadget consists of an organ, which represents a slice of program functionality, an entry point to the organ, and a vein, which represents an execution path that leads to the entry point (Pierazzi et al., 2020). EvadeDroid extracts gadgets from benign apps by identifying entry points, which are typically API calls, through string analysis. The proposed attack assumes that the benign apps used for gadget extraction are not obfuscated, particularly in terms of their API calls. This is because EvadeDroid relies on string analysis to identify entry points, which limits its ability to extract gadgets from obfuscated apps. The gadget injection is considered successful when both the classification loss value of the manipulated app increases and the injected adversarial payload conforms to the predefined size of the adversarial payload. Additionally, the injected gadgets are placed within the block of an obfuscated condition statement that is always evaluated as `False` during runtime and cannot be resolved during design time.

Defender's Capabilities. In this study, we assume that the target ML models do not employ adaptive defenses that are aware of the operations performed by EvadeDroid due to disclosing detectors' vulnerability to EvadeDroid. Specifically, these target models are unable to enhance their resilience by incorporating AEs generated by EvadeDroid during adversarial training. Furthermore, they lack the capability to detect and block queries from EvadeDroid if they become suspicious of its origin. Importantly, our analysis suggests that EvadeDroid can still be effective even if we relax the second assumption regarding the defender's capabilities. This is supported by empirical evidence demonstrating that our attack often requires only a minimal number of queries to generate AEs.

4.2. Problem definition

Suppose $\phi : Z \to X \subset \mathbb{R}^n$ is a feature mapping that encodes an input object $z \in Z$ to a feature vector $x \in X$ with dimension n. We denote this as $\phi(Z) = X$. Here, Z represents the input space of Android applications, and X represents the feature space of the app's feature vectors. Furthermore, let $f : X \to \mathbb{R}^2$ and $g : X \times Y \to \mathbb{R}$ denote a malware classifier and its discriminant function, respectively. The function f assigns an Android app $z \in Z$ to a class $f(\phi(z)) = \arg\max_{y=0,1} g_y(\phi(z))$, where $y = 1$ indicates that z is a malware sample and vice versa. The confidence score (soft label) for classifying z into class y is denoted as $g_y(\phi(z))$. Let $T : Z \xrightarrow{\delta \subseteq \Delta} Z$ be a transformation function, denoted as $T_{\delta \subseteq \Delta}(z) = z'$ or simply $T_\delta(z) = z'$, which transforms $z \in Z$ to $z' \in Z$ by applying a sequence of transformations $\delta \subseteq \Delta$ such that z and z' have the same functionality. Here, $\Delta = \delta_1, \delta_2, ..., \delta_n$ represents an action set consisting of safe manipulations (transformations). Each $\delta_i \in \Delta$ can independently preserve the functionality of a malware sample when applied.

In this study, the objective of the proposed evasion attack is to generate an adversarial example $z^* \in Z$ for a given malware app $z \in Z$ by applying a minimal sequence of transformations $\delta \subseteq \Delta$ to the app, using at most Q queries, while ensuring that the amount of injected adversarial payloads is equal to or lower than α. This can be formulated as the following optimization problem:

$$\begin{aligned} \min_{\delta \subseteq \Delta} \quad & |\delta| \\ \text{s.t.} \quad & f(\phi(T_\delta(z))) \neq f(\phi(z)) \\ & q \leq Q \\ & c(T_\delta(z), z) \leq \alpha \end{aligned} \tag{1}$$

where $|\delta|$ denotes the cardinality of δ. Additionally, Q and α represent the evasion cost constraints of EvadeDroid, indicating the maximum query budget and the maximum size of adversarial payloads, respectively. The size of adversarial payloads refers to the relative increase in the size of a malware sample after applying δ, and it is measured using the following payload-size cost function:

$$c(T_\delta(z), z) = \frac{[T_\delta(z)] - [z]}{[z]} \times 100 \tag{2}$$

where [.] represents the size of an APK. Equation (1) can be translated into the following optimization problem to find an optimal subset of transformations in the action set:

$$\begin{aligned} \arg\max_{\delta \subseteq \Delta} \quad & g_{y=0}(\phi(T_\delta(z))) \\ \text{s.t.} \quad & q \leq Q \\ & c(T_\delta(z), z) \leq \alpha \end{aligned} \tag{3}$$

Equation (3) outlines our objective to identify an optimal subset of problem-space transformations δ within the action set Δ that leads to misclassification. Specifically, the optimization aims to enhance the confidence score of classifiers in classifying $\phi(T_\delta(z))$, the feature representation of z modified by applying δ, towards the benign class indicated by 0. Note that the optimization solver is tasked with identifying the optimal δ with a maximum of Q queries, given that the adversarial payloads do not alter the size of z beyond α.

4.3. Methodology

The primary goal of EvadeDroid is to transform a malware app into an adversarial app in such a way that it retains its malicious behavior but is no longer classified as malware by ML-based malware detectors. This is achieved through an iterative and incremental algorithm employed in the proposed attack, which aims to disguise malware APKs as benign ones. The attack algorithm generates real-world AEs from malware apps using *problem-space transformations* that satisfy problem-space constraints. These transformations are extracted from benign apps in the wild, which are similar to malware apps using an *n-gram*-based similarity. In this approach, a *random search* algorithm is used to optimize the manipulations of apps. Each malware app undergoes incremental manipulation during the optimization process, where a sequence of transformations is applied in different iterations. Before delving into the details of the methodology, we offer a brief overview of n-grams and random search.

***n*-Grams** are contiguous overlapping sub-strings of items (e.g., letters or opcodes) with a length of n from the given samples (e.g., texts or programs). This technique captures the frequencies or existence of a unique sequence of items with a length of n in a given sample. In the area of malware detection, several studies have used n-grams to extract features from malware samples (Fuyong and Tiezhu, 2017; Jain and Meena, 2011; Moskovitch et al., 2008; Santos et al., 2013; Shabtai et al., 2012). These features can be either byte sequences extracted from binary content or opcodes extracted from source codes. *n*-Grams opcode analysis is one of the static analysis approaches for detecting Android malware that has been investigated in various related works (Canfora et al., 2015; Islam et al., 2020; Jerome et al., 2014; Mas'ud et al., 2016; Varsha et al., 2017). To conduct such an analysis, the DEX file of an APK is disassembled into smali files. Each smali file corresponds to a specific class in the source code of the APK that contains variables, functions, etc. *n*-Grams are extracted from the opcode sequences that appear in different functions of the smali files.

Random Search (RS) (Rastrigin, 1963) is a simple yet highly exploratory search strategy that is used in some optimization problems to find an optimal solution. It relies entirely on randomness, which means RS does not require any assumptions about the details of the objective function or transfer knowledge (e.g., the last obtained solution) from one iteration to another. In the general RS algorithm, the sampling distribution S and the initial candidate solution $x^{(0)}$ are defined based on the feasible solutions of the optimization problem. Then, in each iteration t, a solution $x^{(t)}$ is randomly generated from S and evaluated using an objective function regarding $x^{(t-1)}$. This process continues through different iterations until the best solution is found or the termination conditions are met. It's noteworthy that RS can be a search strategy with high query efficiency in generating AEs (Croce et al., 2022).

The workflow of the attack pipeline is illustrated in Fig. 1, which consists of two phases: (i) preparation and (ii) manipulation.

4.3.1. Preparation

The primary objective of this step is to construct an action set comprising a collection of safe transformations that can directly manipulate Android applications. Each transformation in the action set should be capable of altering APKs without causing crashes while preserving their functionality. In this study, program slicing (Weiser, 1984), implemented in Pierazzi et al. (2020), is utilized to extract the gadgets that make up the transformations collected in the action set. During the preparation step, two important considerations are determining appropriate donors and identifying suitable gadgets. Employing effective gadgets enables the modification of a set of features that can alter the classifier's decision. EvadeDroid achieves this by executing the following two sequential steps:

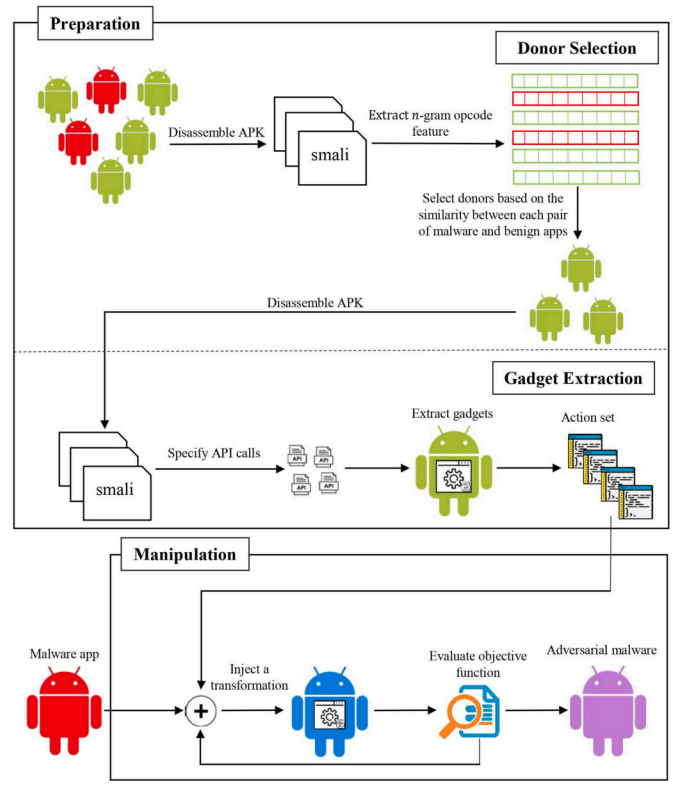

Fig. 1. Overview of EvadeDroid's pipeline.

a) Donor selection. EvadeDroid selects donors from a pool of benign apps in order to mimic malware instances as benign ones. While it is possible to extract gadgets from any available benign app, collecting transformations from a large corpus of apps is computationally expensive due to the complexity of the program-slicing technique used for organ harvesting. Additionally, identifying potential donors resembling malware apps can lead to obtaining transformations that facilitate disguising malware apps as benign. This is because malware apps that share similarities with benign ones may require fewer transformations to become AEs. In this study, EvadeDroid adopts a strategy of limiting the number of donors, i.e., choosing donors from the pool of benign apps that resemble malware apps. Our empirical results demonstrate that utilizing transformations from such benign apps accelerates the process of converting malware apps into benign ones, resulting in a reduced number of queries and transformations required for manipulation (refer to Appendix B for more details).

More specifically, by utilizing the extracted gadgets from these donors, EvadeDroid can generate effective adversarial perturbations by considering both feature and learning vulnerabilities (Maiorca et al., 2020; Muñoz-González and Lupu, 2019). Fig. 2 provides a conceptual representation of EvadeDroid's performance in evading the target classifier. As depicted in Fig. 2, incorporating segments of benign apps that resemble malware apps can either make malware apps look benign ($T_\delta(z) = z_1^*$ where $\delta = \{\delta_1, \delta_2, \delta_3\}$), or shift them towards the blind spots

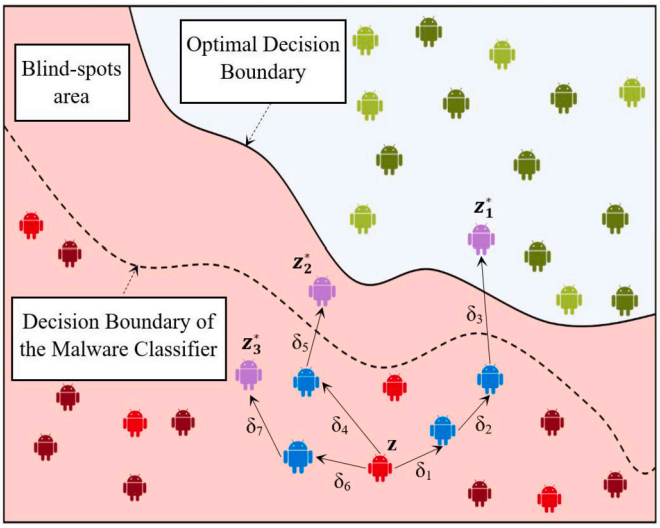

Fig. 2. The functionality of EvadeDroid in generating real-world adversarial malware apps. The dark red and dark green samples are, respectively, the inaccessible malware and benign samples that have been used for training the malware classifier. Light red and light green samples represent, respectively, accessible malware and benign samples in the wild. The blue and purple samples are manipulated malware apps and AEs, respectively.

of the target classifier (e.g., $T_\delta(z) = z_2^*$ where $\delta = \{\delta_4, \delta_5\}$). Note that some sequences of transformations may fail to generate successful AEs (e.g., $\{\delta_6, \delta_7\}$). In this work, we employ an n-gram-based opcode technique to assess the similarities between malware and benign samples. Extracting n-gram opcode features enables automated feature extraction from raw bytecodes, allowing EvadeDroid to measure the similarity between real objects without requiring knowledge of the feature vector of Android apps in the feature space of the target black-box malware classifiers. We extract n-grams following typical approaches found in the literature (e.g., Kang et al., 2016; Ko et al., 2013), but with a focus on opcode types rather than the opcodes themselves. The n-gram opcode feature extraction utilized in this study involves the following main steps:

1. Disassemble Android application's DEX files into smali files using Apktool.
2. Discard operands and extract n-grams from the types of all opcode sequences in each smali file belonging to the app. For example, consider a sequence of opcodes in a smali file: *I: if-eq M: move G: goto I: if-ne M: move-exception G: goto/16 M: move-result*. In this case, we have 7 opcodes with 3 types (i.e., I, M, G). Note *IM, MG, GI, GM* are all unique 2-grams that appeared in the given sequence.
3. Map the extracted feature sets to a feature space H by aggregating all observable n-grams from all APKs.
4. Create a feature vector $h \in H$ for each app, where each element of h indicates the presence or absence of a specific n-gram in the app.

Suppose M and B represent the sets of malware and benign apps, respectively, available to EvadeDroid. The similarity between each pair of a malware app $m_i \in M$ and a benign app $b_j \in B$ is determined by measuring the containment (Kang et al., 2016; Ko et al., 2013) of b_j in m_i using the following approach:

$$\sigma(m_i, b_j) = \frac{|v(m_i) \cap v(b_j)|}{|v(b_j)|} \tag{4}$$

where $v(m_i)$ and $v(b_j)$ represent the sets of features with values of 1 in h_{m_i} and h_{b_j}, respectively, and $|.|$ denotes the number of features. Specifically, $|v(m_i) \cap v(b_j)|$ denotes the number of common features between m_i and b_j. It is worth emphasizing that most Android malware apps are created using repackaging techniques, where attackers disguise malicious payloads in legitimate apps (Chen et al., 2015). Therefore, we consider the containment of benign samples in malware samples to determine the similarities between each pair of malware and benign samples. To identify suitable donors, we calculate a weight for each benign app $b_i \in B$ according to equation (4):

$$w_{b_j} = \frac{\sum_{\forall m_i \in M} \sigma(m_i, b_j)}{|M|} \tag{5}$$

where $|M|$ represents the number of malware apps. We then sort the benign apps in descending order based on their corresponding weights. Finally, we select the top-k benign apps as suitable donors for gadget extraction. Note that w_{b_j} reflects how closely b_j aligns with the distribution of malware apps, offering a measure of its resemblance to the characteristics of malware.

b) Gadget extraction. We collect gadgets based on the desired functionality we aim to extract from donors. EvadeDroid intends to simulate malware samples to benign ones from the perspective of static analysis; therefore, the payloads responsible for the key semantics of donors are proper candidates for extraction. To access the semantics of Android applications, EvadeDroid extracts the payloads containing API calls (i.e., the code snippet encompassing an API call and all its associations) since API calls represent the main semantics of apps (Aafer et al., 2013; Sami et al., 2010). An API call is an appropriate point in the bytecode of an APK because the snippets encompassing the API calls are related to one of the app semantics. In sum, gadget extraction from donors consists of the following main steps:

1. Disassemble DEX files of donors into smali files by using Apktool.
2. Perform string analysis on each app to identify all API calls in its smali files.
3. Extract the gadgets associated with the collected API calls from each app.

Ultimately, the action set Δ is formed by taking the union of the extracted gadgets.

4.3.2. Manipulation

We employ Random Search (RS) as a simple black-box optimization method to solve equation (3). Specifically, for each malware sample z, EvadeDroid utilizes RS to find an optimal subset of transformations δ in order to generate an adversarial example z^*. RS offers a significant advantage in terms of query reduction compared to other heuristic optimization algorithms, such as Genetic Algorithms (GAs). This is because RS only requires one query in each iteration to evaluate the current solution. Algorithm 1 outlines the key steps of the manipulation component in the proposed problem-space evasion attack. As depicted in Algorithm 1, the RS method randomly selects a transformation λ from the action set Δ to generate z^* for z. Subsequently, based on the adversarial payload size α, the algorithm applies λ to z only if it can improve the objective function L defined in equation (3), which corresponds to the discriminant function of the target classifier for $y = 0$.

Hard-label Setting. In Algorithm 1, we assume that our attack has access to the soft label of the target classifier. This means that EvadeDroid can obtain the confidence score provided by the black-box classification model when making queries. However, in real-world scenarios, such as antivirus systems, the target classifier may only provide hard labels (i.e., classification labels) for Android apps. In this study, we consider two approaches, namely optimal and non-optimal hard-label attacks, to address this challenge. In the optimal hard-label attack, the adversary aims to generate AEs by applying minimal transformations. To achieve this, EvadeDroid modifies the objective function of the proposed RS algorithm (i.e., equation (3)) by maximizing the following objective function, while considering the evasion cost:

Algorithm 1: Generating a real-world adversarial example.

Input: z, the original malware sample; Δ, the action set; L, the objective function; ϕ, the feature mapping function; c, the payload-size cost function; Q, the query budget; α, the allowed adversarial payload size.
Output: z^*, an adversarial example; δ, an optimal transformations.

```
q ← 1 ;
z* ← z;
L_best ←-∞;
δ ← ∅;
while q ≤ Q and z* is classified as a malware do
    λ ← Select a transformation randomly from Δ \δ;
    z' ← T_λ(z*);
    l = L(ϕ(z'));
    if c(z, z') ≤ α then
        if L_best ≤ l then
            L_best ← l;
            z* ← z';
            δ ← δ ∪ λ
        end
    end
end
return z*, δ
```

$$\begin{aligned} \arg\max_{\delta \subseteq \Delta} \quad & s(T_\delta(z)) \\ \text{s.t.} \quad & q \le Q \\ & c(T_\delta(z), z) \le \alpha \end{aligned} \tag{6}$$

where Q (i.e., number of queries) and α (i.e., size of adversarial payloads) represent evasion cost budgets, and c denotes the payload-size cost function (equation (2)). Moreover, s is the following similarity function:

$$s(a) = \max_{\forall b \in B} \frac{|v(a) \cap v(b)|}{\|h_a - h_b\|_1} \tag{7}$$

where B represents all available benign apps in the wild. $v(a)$ and $v(b)$ represent the sets of features with values of 1 in h_a (i.e., the feature vector of a) and h_b (i.e., the feature vector of b), respectively, and $|.|$ denotes the number of features. Furthermore, $\|h_a - h_b\|_1$ denotes the sum of the absolute differences (i.e., l_1-norm) between the opcode-based feature vectors of a and b. The l_1-norm enhances the accuracy of our similarity measurement, particularly in scenarios where the number of common features between various pairs of malware samples and benign samples is the same, aiding EvadeDroid in identifying the maximum similarity. Note that equation (7) aims to measure the similarity between two apps based on not only a large set of common features but also a small distance. The underlying idea behind the introduced objective function is rooted in our primary approach to misleading malware classifiers. In other words, a transformation can be applied to a malware app if it maintains or increases the maximum similarity between the malware app and available benign apps.

On the other hand, in the non-optimal hard-label attack, EvadeDroid applies random transformations to malware until it creates an AEs or reaches the predefined query budget. Specifically, in this setting, EvadeDroid randomly selects and applies a transformation from the action set to the malware app in each query. The target classifier is then queried to determine the label of the modified app. If the label indicates that the app is still classified as malware, EvadeDroid repeats this process.

It is important to highlight that Fig. 3 depicts the procedure of manipulating an Android malware app through a problem-space transformation, specifically injecting an extracted gadget into a malware app. For more detailed information on the implementation of EvadeDroid, we refer the reader to Appendix C.

5. Simulation results

In this section, we empirically assess the performance of EvadeDroid in deceiving various academic and commercial malware classifiers. Our experiments aim to answer the following research questions:

RQ1. How does the evasion cost affect the performance of EvadeDroid? (§5.2)
RQ2. Is EvadeDroid a versatile attack that can evade different Android malware detectors without relying on any specific assumptions? (§5.2)
RQ3. How does the performance of EvadeDroid compare to other similar attacks? (§5.3)
RQ4. Is EvadeDroid applicable in real-world scenarios? (§5.4)
RQ5. How does EvadeDroid demonstrate its performance despite the restriction of not being able to query the target detectors? (§5.5)
RQ6. How does the proposed RS-based manipulation strategy affect the performance of EvadeDroid? (§5.6)

All experiments have been run on a Debian Linux workstation with an Intel (R) Core (TM) i7-4770 K, CPU 3.50 GHz, and 32 GB RAM.

5.1. Experimental setup

Here, we provide an overview of the target detectors, datasets, and evaluation metrics we consider in our experiments.

5.1.1. Target detectors

To ensure that our conclusions are not limited to a specific type of malware detection, we evaluate EvadeDroid against various malware detectors to demonstrate the effectiveness of the proposed attack. In particular, our evaluation focuses on assessing EvadeDroid's performance against well-known Android malware detection models, namely DREBIN (Arp et al., 2014), Sec-SVM (Demontis et al., 2017), ADE-MA (Li and Li, 2020), MaMaDroid (Onwuzurike et al., 2019), and Opcode-SVM (Jerome et al., 2014). These models have been extensively studied in the context of detecting problem-space adversarial attacks in the Android domain (Chen et al., 2019; Grosse et al., 2017; Li et al., 2021b; Pierazzi et al., 2020; Zhang et al., 2021). For more details about these detectors, please refer to Appendix D.

5.1.2. Dataset

We evaluate the performance of EvadeDroid using the dataset provided in Pierazzi et al. (2020). This dataset consists of $\approx 170K$ samples, each represented using the DREBIN (Arp et al., 2014) feature set. The samples are feature representations of Android apps collected from AndroZoo (Allix et al., 2016) and labeled by Pierazzi et al. (2020) using a threshold-based labeling approach. These collected apps were published between January 2017 and December 2018. According to the labeling criteria in Pierazzi et al. (2020), an APK is considered malicious or clean if it has been detected by any 4+ or 0 VirusTotal (VT) (VirusTotal, 2004) engines, respectively. It is important to note that the threshold-based labeling approach does not rely on specific engines but considers the number of engines involved (Salem, 2021). Therefore, the engines used for labeling may vary from sample to sample.

Table 2 presents the specifications of datasets utilized in our research where their samples were randomly chosen from the collected data provided in Pierazzi et al. (2020). It's worth mentioning that there is no overlap between the inaccessible and accessible datasets. EvadeDroid exclusively makes use of the accessible dataset, which comprises $2K$ benign samples for donor selection and $1K$ malware samples for the creation of AEs. To fulfill the requirement of direct utilization of apps in our problem-space attack, we collect $3K$ apps corresponding to EvadeDroid's accessible samples from AndroZoo, based on the apps' specifications provided with the dataset (Pierazzi et al., 2020). In this study, we employ two training sets with different scales (i.e., $12K$ and $100K$) for training classifiers. The proportion between benign and malware samples in the training sets is chosen to avoid spatial dataset bias (Pendlebury et al., 2019). Fig. 4 illustrates the temporal distribution of the smaller training set, demonstrating the absence of temporal bias as these apps were published across various months. The larger training set follows a similar distribution. In §5.2, §5.3, and §5.5, a training set with a reasonable size (i.e., $12K$) is used due to the time-consuming preprocessing required by the apps in the MaMaDroid and

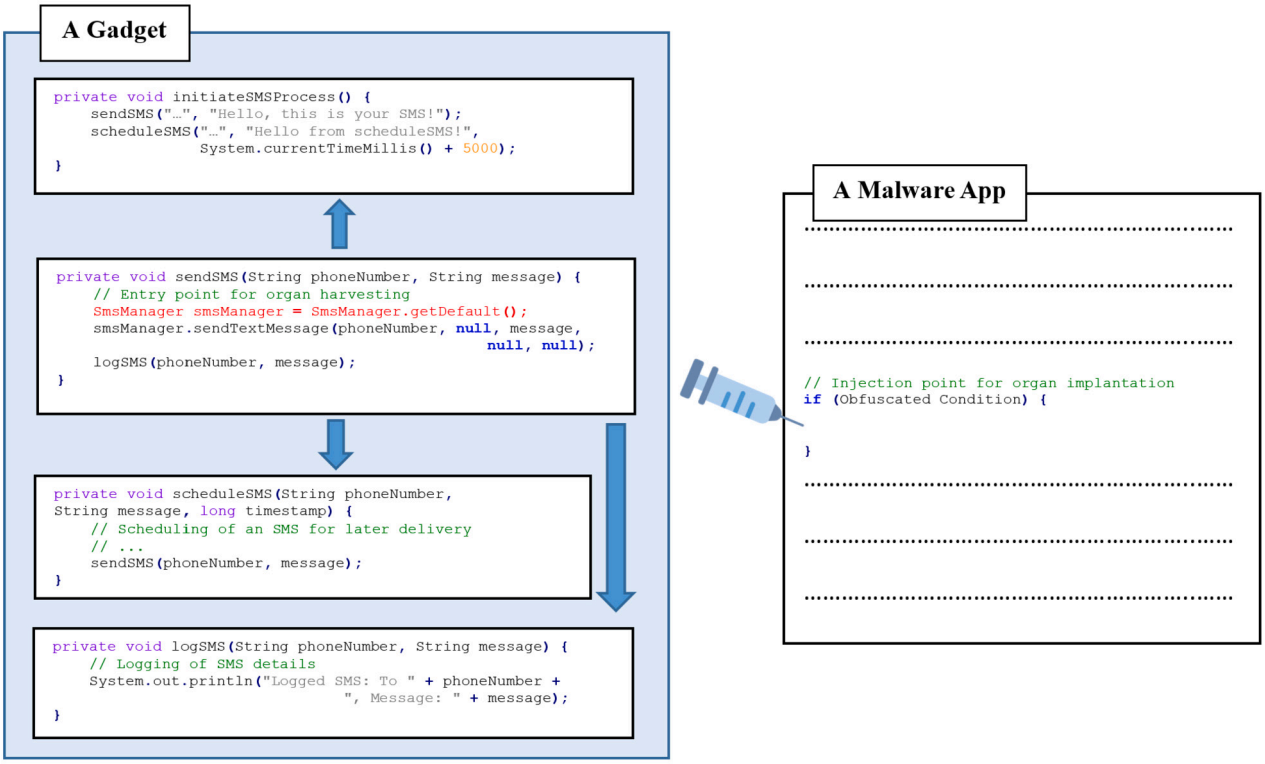

Fig. 3. Applying a problem-space transformation (i.e., gadget) into a malware app involves injecting the gadget extracted from an API call entry point (`e.g.`, `SmsManager`) in a donor into an obfuscated false condition statement within the malware app. The code snippets are displayed in `Java` representation to facilitate better understanding.

Table 2
Datasets used in our experiments.

Dataset	No. of Benign samples	No. of Malware Samples	Relevant Experiment
Inaccessible Dataset (Training Samples)	10 K	2 K	§5.2, §5.3 ,§5.5
	90 K	10 K	§5.5
Accessible Dataset (EvadeDroid's samples)	2 K	1 K	All

Opcode-SVM, especially the former. Note that MaMaDroid and Opcode-SVM employ their own distinct feature representations, which differ from the DREBIN feature representation used in Pierazzi et al. (2020). Therefore, to provide the training set for these detectors, we have to directly collect all considered apps in the training set from AndroZoo based on the specifications provided by Pierazzi et al. (2020). Subsequently, the apps are embedded in the MaMaDroid and Opcode-SVM feature spaces using a feature extraction method. In the second evaluation conducted in §5.5, we employ a larger training set (incl., $100K$ samples) to train DREBIN and Sec-SVM in order to illustrate the impact of a larger training set on EvadeDroid. It is important to highlight that our empirical evaluation shows that training classifiers with more samples does not significantly alter the performance of EvadeDroid.

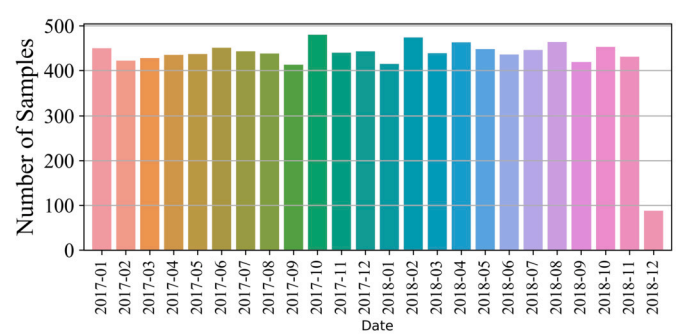

Fig. 4. The temporal distribution of training samples. The dataset (Pierazzi et al., 2020) lacked clarity regarding the release dates of the $\approx 1.5K$ samples in our training set.

5.1.3. Evaluation metrics

We utilize the True Positive Rate (TPR) and False Positive Rate (FPR) as performance metrics for evaluating the effectiveness of malware classifiers in detecting Android malware. In Fig. 5, we present the Receiver Operating Characteristic (ROC) curves of DREBIN, Sec-SVM, ADE-MA, MaMaDroid, and Opcode-SVM, the Android malware detectors used in this study, on the $12K$ training samples in the absence of our proposed attack. Note that the ROC curves were generated using 10-fold cross-validation. In addition to these metrics, we introduce the Evasion Rate (ER) and Evasion Time (ET) as EvadeDroid's performance assessment metrics in deceiving malware classifiers. ER is calculated as the ratio of correctly detected malware samples that are able to evade the target classifiers after manipulation to the total number of correctly classified malware samples. ET represents the average time, expressed in seconds, required by EvadeDroid to generate an AE, encompassing both optimization and query times. Note that the optimization time primarily consists of the execution times of random search, injecting problem-

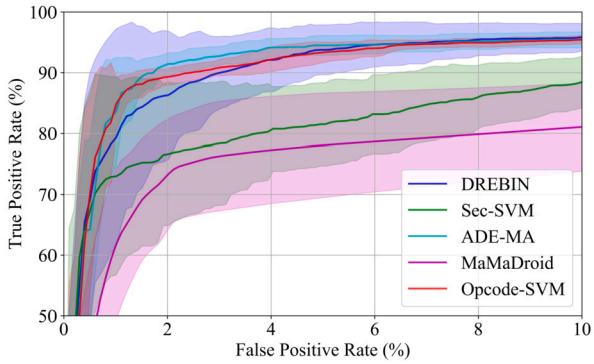

Fig. 5. ROC curves of DREBIN, Sec-SVM, ADE-MA, MaMaDroid, and Opcode-SVM in the absence of EvadeDroid. The regions with translucent colors that encompass the lines are standard deviations.

space transformations, and performing feature extraction to represent manipulated apps within the feature space. Further details of our experimental settings can be found in Appendix E.

5.2. Evasion costs and generalizability

This section first examines the influence of the allowed adversarial payload size α and the query budget Q on the performance of EvadeDroid to answer **RQ1**. Specifically, the evasion rates of EvadeDroid in fooling various malware detectors under different adversarial payload sizes and query numbers are depicted in Fig. 6. Fig. 6a demonstrates that the evasion rate is influenced by the size of the adversarial payload, as increasing the size allows EvadeDroid to modify more malware applications. However, we observed that for $\alpha \geq 30\%$, the impact on the evasion rate becomes less significant, as most sequences of viable transformations almost reach a plateau at $\alpha = 30\%$. Furthermore, no further improvement in evasion rates is observed beyond $\alpha = 50\%$. In addition to the adversarial payload size, the query budget is another constraint that affects the evasion rate of EvadeDroid. Fig. 6b presents a comparison of the effect of different query numbers on the evasion rates of EvadeDroid against various malware detectors, with an allowed adversarial payload size of $\alpha = 50\%$. As can be seen in Fig. 6b, EvadeDroid requires a larger number of queries to generate successful AEs for bypassing Sec-SVM as compared to other detectors. This can be attributed to the fact that Sec-SVM, being a sparse classification model, relies on a greater number of features for malware classification compared to other classifiers. Consequently, EvadeDroid needs to apply more transformations to malware apps in order to deceive this more resilient variant of DREBIN. Additionally, Fig. 6b demonstrates that a query budget of $Q = 20$ is nearly sufficient for EvadeDroid to achieve maximum evasion rate when attempting to bypass a malware detector. It is important to highlight that for the remaining experiments of the paper, we have chosen to use $Q = 20$ and $\alpha = 50\%$ as they yield the optimal performance for EvadeDroid.

To answer **RQ2**, we conduct an experiment involving various malware detectors and different attack settings. Specifically, we include DREBIN, SecSVM, ADE-MA, MaMaDroid, and Opcode-SVM to cover different ML algorithms (i.e., linear vs. non-linear malware classifiers, and gradient-based vs. non-gradient-based malware classifiers) and diverse features (i.e., discrete vs. continuous features, and syntax vs. opcode vs. semantic features). Additionally, we explore different attack settings (soft label vs. hard label) to demonstrate EvadeDroid's adaptability in various scenarios. The performance of the proposed attacks under different settings and malware detectors is presented in Table 3. As shown in this table, EvadeDroid demonstrates effective evasion capabilities against various malware detectors, including DREBIN, Sec-SVM, and ADE-MA with syntax binary features, as well as MaMaDroid with semantic continuous features and Opcode-SVM with opcode binary features. The evaluation also reveals that EvadeDroid performs similarly

Table 3
Effectiveness of EvadeDroid in misleading different malware detectors when $Q = 20$ and $\alpha = 50\%$. NoQ, NoT, and AS denote Avg. No. of Queries, Avg. No. of Transformations, and Avg. Adversarial Payload Size, respectively.

Type of Threat	Target Model	ER (%)	ET (s)	NoQ	NoT	AS (%)
Soft Label	DREBIN	88.9	210.3	3	2	15.5
	Sec-SVM	85.1	495.4	9	4	16.4
	ADE-MA	86.0	126.2	2	1	16.3
	MaMaDroid	94.8	131.4	1	1	15.9
	Opcode-SVM	79.6	114.1	3	2	18.3
Optimal Hard Label	DREBIN	84.5	240.6	4	2	16.2
	Sec-SVM	82.6	613.1	9	6	16.5
	ADE-MA	84.4	121.2	2	1	16.3
	MaMaDroid	94.8	133.7	1	1	15.9
	Opcode-SVM	74.1	101.2	2	1	18.2
Non-optimal Hard Label	DREBIN	79.7	357.2	4	4	16.9
	Sec-SVM	78.2	782.8	9	9	17.3
	ADE-MA	82.7	157.3	2	2	16.4
	MaMaDroid	94.8	132.6	1	1	15.9
	Opcode-SVM	66.6	76.2	1	1	18.3

well in the optimal hard-label setting compared to the soft-label setting. It is important to note that the comparison between soft-label attacking and non-optimal hard-label attacking highlights the influence of optimizing manipulations on the performance of EvadeDroid against different detectors. While only applying transformations to malware apps is sufficient for MaMaDroid, optimizing manipulations can enhance EvadeDroid's effectiveness against other detectors, especially Opcode-SVM. For instance, our findings shown in Table 3 demonstrate a 13% improvement in the ER of EvadeDroid when targeting Opcode-SVM in the soft-label setting, compared to the non-optimal hard-label setting. Furthermore, when operating in the soft-label setting, EvadeDroid requires notably fewer transformations to bypass DREBIN and Sec-SVM, as compared to the non-optimal hard-label setting (e.g., 4 vs. 9 for Sec-SVM), which confirms the effectiveness of EvadeDroid in solving the optimization problem defined in equation (1). Table 3 further illustrates that our optimization leads to a substantial reduction in ET compared to the non-optimal hard-label setting. Specifically, for DREBIN and Sec-SVM, this leads to a time reduction of $\approx 41\%$ and $\approx 37\%$, respectively. This significant enhancement can be attributed to the reduction in the number of transformations, achieved through the utilization of our proposed optimization technique. Note that ET brings attention to the varying time overheads associated with the feature extraction process used to compute objective values when attacking different target detectors. For example, while NoQ and NoT are the same in attacking MaMaDroid and Opcode-SVM in the non-optimal hard-label setting, the ET for MaMaDroid is significantly higher than that for Opcode-SVM. The observed distinction is rooted in the considerable time consumption of the feature extraction process in MaMaDroid.

In summary, the results demonstrate that the proposed adversarial attack is a versatile black-box attack that does not make assumptions about target detectors, including the ML algorithms or the features used for malware detection. Furthermore, it can operate effectively in various attack settings.

5.3. EvadeDroid vs. other attacks

To answer **RQ3**, we conduct an empirical analysis to assess how EvadeDroid performs in comparison to other similar attacks. To establish a comprehensive evaluation of EvadeDroid, we consider four baseline attacks: PiAttack (Pierazzi et al., 2020), Sparse-RS (Croce et al., 2022), ShadowDroid (Zhang et al., 2021), and GenDroid (Xu et al., 2023) operating in white-box, gray-box, semi-black-box, and black-box settings, respectively. These attacks serve as suitable benchmarks, allowing us to assess the performance of EvadeDroid from different perspectives, such as evasion rate and the number of queries. Similar

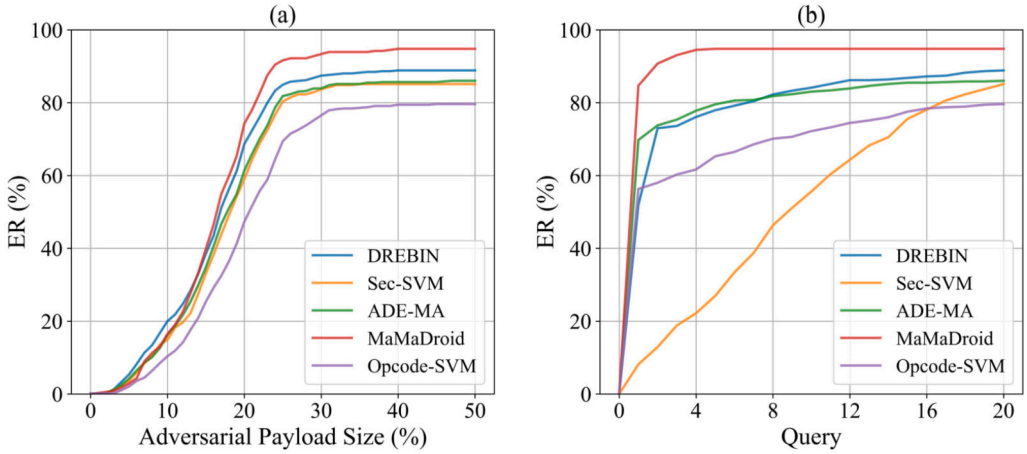

Fig. 6. ERs of EvadeDroid operating in the soft-label setting in deceiving different Android malware detectors in terms of (a) different adversarial payload sizes and (b) different queries.

Table 4
ERs of EvadeDroid, PiAttack, Sparse-RS, ShadowDroid, and GenDroid in misleading DREBIN, Sec-SVM, and ADE-MA. NoQ denotes Avg. No. of Queries.

Target Model	Evasion Attach	ER (%)	NoQ
DREBIN	EvadeDroid	88.9	3
	PiAttack	99.6	N/A
	Sparse-RS	18.3	195
	ShadowDroid	95.3	31
	GenDroid	95.5	93
Sec-SVM	EvadeDroid	85.1	9
	PiAttack	94.3	N/A
	Sparse-RS	0.4	38
	ShadowDroid	8.6	64
	GenDroid	14.5	336
ADE-MA	EvadeDroid	86.0	2
	PiAttack	100	N/A
	Sparse-RS	99.7	2
	ShadowDroid	77.8	29
	GenDroid	100	81

to EvadeDroid, Sparse-RS, ShadowDroid, and GenDroid generate AEs by querying the target detectors. Additionally, PiAttack is a problem-space adversarial attack that employs a similar type of transformation to generate AEs. Although PiAttack is a white-box evasion attack, it establishes a benchmark for optimal evasion performance, facilitating the evaluation of the comparative effectiveness of other attacks with limited or zero knowledge about the targeted detectors. For further information about these attacks, please refer to Appendix F. In this experiment, we chose DREBIN, Sec-SVM, and ADE-MA as the target detectors because they align with the threat models of PiAttack, Sparse-RS, and ShadowDroid. Table 4 shows the ERs of different adversarial attacks in deceiving various malware detectors. As can be seen in Table 4, although EvadeDroid has zero knowledge about DREBIN, Sec-SVM, and ADE-MA, its evasion rates for bypassing these detectors are comparable to PiAttack, where the adversary has full knowledge of the target detectors. Moreover, our empirical analysis shows that EvadeDroid requires adding more features to evade DREBIN, Sec-SVM, and ADE-MA. In concrete, on average, EvadeDroid makes 54–90 new features appear in the feature representations of the malware apps when it applies transformations to the apps for evading DREBIN, Sec-SVM, and ADE-MA, while the transformations used by PiAttack on average, trigger 11–68 features. PiAttack's ability to add a smaller number of features is attributed to its complete knowledge of the details of DREBIN, Sec-SVM, and ADE-MA. However, EvadeDroid lacks this specific information.

Furthermore, as shown in Table 4, the evasion rate of Sparse-RS for DREBIN and Sec-SVM demonstrates that random alterations in malware features do not necessarily result in the successful generation of AEs, even when adversaries have access to the target models' training set. Although EvadeDroid operates solely in a black-box setting, this attack outperforms Sparse-RS by a considerable margin for both DREBIN and Sec-SVM, i.e., 70.6% and 84.7% improvement, respectively. Moreover, EvadeDroid considerably surpasses ShadowDroid in attacking Sec-SVM and ADE-ME. Especially, in contrast to EvadeDroid, ShadowDroid is unsuccessful in effectively evading Sec-SVM, which is a robust detector against AEs. Note that the superior performance of ShadowDroid compared to EvadeDroid in bypassing DREBIN is based on the assumption that target detectors primarily rely on API calls and permissions. However, this assumption is not practical in real scenarios, as detectors may employ other features for malware detection. Table 4 further illustrates that GenDroid exhibits superior evasion rates compared to EvadeDroid when targeting DREBIN and ADE-MA; nevertheless, its efficacy is substantially nullified when facing Sec-SVM, a resilient malware detector. Our empirical analysis also highlights the remarkable efficiency of EvadeDroid in terms of the number of queries compared to other query-based attacks. Specifically, on average, EvadeDroid requires only 2–9 queries to bypass DREBIN, Sec-SVM, and ADE-MA, while Sparse-RS, ShadowDroid, and GenDroid demand 2–195, 29–64, and 81–336 queries, respectively.

In summary, the experimental results validate the practicality of EvadeDroid, which adopts a realistic threat model, in comparison to other attacks for generating AEs. Specifically, the threat models of PiAttack and Sparse-RS are essentially proposed for the detectors that operate in the DREBIN feature space, but their threat models are not practical for targeting detectors like MaMaDroid. Furthermore, ShadowDroid's effectiveness is limited to scenarios where malware detection is solely based on API calls and permissions. For instance, as demonstrated in Zhang et al. (2021), ShadowDroid is unable to deceive MaMaDroid or opcode-based detectors. In contrast, as shown in §5.2, EvadeDroid is capable of effectively fooling these types of detectors as its problem-space transformations are independent of feature space. Additionally, although GenDroid operates in the ZK setting, it perhaps encounters challenges in evading robust malware detectors like Sec-SVM and might pose potential issues in real-world scenarios due to the substantial number of queries it requires compared to EvadeDroid. Finally, Sparse-RS, ShadowDroid, and GenDroid might not be deemed realistic approaches as their abilities to satisfy problem-space constraints, particularly robustness-to-preprocessing and plausibility constraints, are questionable.

Table 5
Performance of EvadeDroid in the hard-label setting on five commercial antivirus products. NoM denotes No. of Detected Malware by each engine among 100 malware apps.

Engine	NoM	EvadeDroid			
		ER (%)	Avg. Attack Time (s)	Avg. No. of Queries	Avg. Query Time
AV1	54	68.5	31.3	1	214.3
AV2	32	87.5	54.7	2	387.2
AV3	31	74.2	124.1	2	446.6
AV4	41	100	35.2	1	329.7
AV5	11	63.6	21.5	1	272.9

5.4. EvadeDroid in real-world scenarios

This experiment aims to investigate **RQ4** to demonstrate the practicality of EvadeDroid in real-world scenarios. Although the ability of EvadeDroid in the hard-label setting indicates that this attack can transfer to real life, we further consolidate this observation by measuring the impact of EvadeDroid on commercial antivirus products that are available on VT to confirm the practicality of our proposed attack in real scenarios. We chose five popular antivirus engines in the Android ecosystem based on the recent ratings of the endpoint protection platforms reported by AV-Test (AV-TEST, 2004). They are the top AVs in AV-Test capable of detecting malware apps in EvadeDroid's accessible dataset. Moreover, 100 malware apps belonging to different malware families have been randomly selected from the $1K$ malware apps available to EvadeDroid to evaluate the performance of this attack on the aforementioned five commercial detectors. To ensure the reliability of our experiment, it is crucial to confirm that the labels assigned to the malware apps used in this experiment have remained consistent. This is because the labels of collected apps are based on their corresponding samples in our benchmark dataset (Pierazzi et al., 2020), while the labels assigned by antivirus engines to apps can potentially change over time. Therefore, we meticulously selected 100 apps that are still malware based on the threshold labeling criteria used in our primary dataset at the time of our experiment, i.e. on September 11, 2022, through querying VT. Furthermore, for each antivirus product, we generate AEs for the apps detected as malware by the antivirus. Table 5 presents the results of the experiment in which EvadeDroid attempts to deceive each AV in the optimal hard-label setting. In this experiment, we have assumed $Q = 10$ and $\alpha = 50\%$. As can be seen in Table 5, our proposed attack can effectively evade all antivirus products with a few queries. Here the effectiveness of EvadeDroid can be primarily attributed to the transformations rather than the optimization technique. This is evident from the fact that in most cases, only one query is required to generate AEs. We further investigate the performance of EvadeDroid against the overall effect of VT. Fig. 7 shows the average number of VT detections for all 100 malware apps after each attempt of EvadeDroid to change malware apps into AEs. As depicted in Fig. 7, EvadeDroid can effectively deceive VT engines with an average of 70.67%. It is worth noting that the findings in this experiment validate the results observed in previous studies (e.g., Ceschin et al., 2019).

Responsible Disclosure. We conducted a responsible disclosure process to ensure the security community was informed of our research findings. As part of this process, we not only reached out to VT but also notified the antivirus engines that were affected by EvadeDroid by providing detailed information about our attack methodology and sharing some test cases.

5.5. Transferable adversarial examples

In general, when decision-based adversarial attacks, such as EvadeDroid, encounter difficulty in querying specific target detectors, they can create transferable AEs using a surrogate classifier. Here we explore

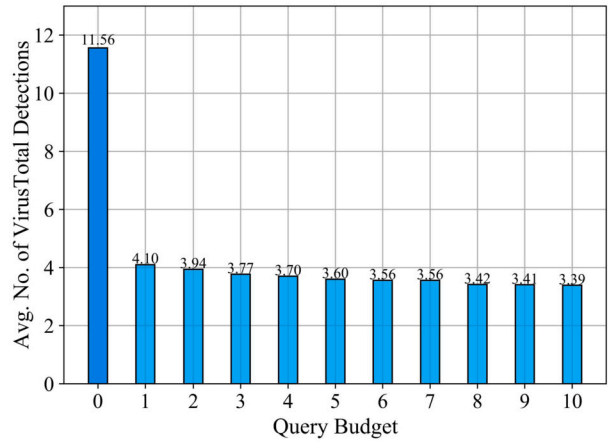

Fig. 7. Performance of EvadeDroid in evading VT engines against different query budgets.

Table 6
Transferability of AEs generated by EvadeDroid.

Surrogate Model	Target Model	ER (%)
DREBIN	Sec-SVM	25.5
	ADE-MA	88.7
	MaMaDroid	63.0
	Opcode-SVM	42.2
Sec-SVM	DREBIN	95.7
	ADE-MA	98.5
	MaMaDroid	95.4
	Opcode-SVM	53.7
ADE-MA	DREBIN	49.3
	Sec-SVM	8.7
	MaMaDroid	67.5
	Opcode-SVM	22.0
MaMaDroid	DREBIN	41.1
	Sec-SVM	6.0
	ADE-MA	88.9
	Opcode-SVM	37.0
Opcode-SVM	DREBIN	32.8
	Sec-SVM	10.9
	ADE-MA	66.8
	MaMaDroid	74.83

RQ5 by considering transferable AEs. To investigate the transferability of EvadeDroid, we evaluate the evasion rates of AEs generated on a model (e.g., Sec-SVM), which works as a surrogate model, in misleading other target models (e.g., DREBIN). This is a stricter threat model that indicates the performance of EvadeDroid in cases where adversaries are not capable of querying the target detectors. Table 6 demonstrates that when EvadeDroid employs a stronger surrogate model (e.g., Sec-SVM), the AEs exhibit higher transferability. Note that the reported ERs in Table 6 are the evasion rates of successful AEs that are also successfully transferred.

We further compare the transferability of EvadeDroid with PiAttack (Pierazzi et al., 2020) as it is similar to ours in terms of transformation type. This attack uses two kinds of primary features for misclassification, and side-effect features for satisfying problem-space constraints to generate realizable adversarial examples. However, EvadeDroid is not constrained by features as it operates in black-box settings. We specifically measure the transferability of the AEs in fooling Sec-SVM when DREBIN is the surrogate model. We ensure that the original apps of the AEs are correctly detected by Sec-SVM. Both DREBIN and Sec-SVM are trained with $100K$ apps (incl., $90K$ benign apps and $10K$ malware apps) to see the effect of large ML models on EvadeDroid's per-

Table 7
RS-based vs. GA-based manipulation strategies in EvadeDroid. NoQ indicates Avg. No. of Queries and NoT denotes Avg. No. of Transformations.

Search Method	ER (%)	ET (s)	NoQ	NoT
RS	88.9	210.3	3	2
GA	65.1	630.7	22	5

formance. The experimental results show that the ERs of the PiAttack and EvadeDroid in circumventing DREBIN are 99.06% and 82.12%, respectively. Furthermore, EvadeDroid is much more transferable as the transferability of the AEs generated by EvadeDroid is 58.05%, while 23.23% for PiAttack.

5.6. The impact of search strategy on EvadeDroid

To answer **RQ6**, we perform an empirical analysis to evaluate the performance of EvadeDroid when utilizing an alternative search strategy for manipulation. Specifically, we introduce a baseline manipulation method based on GA for use in EvadeDroid, where the fitness function of the baseline is the same as the RS-based method. In the proposed GA-based manipulation method, the individuals in the population (representing potential solutions) are binary strings with a length equal to the action set Δ, where 1 indicates the corresponding transformation in Δ should be used for manipulation. This approach enhances the solution across various generations. In this experiment, the query budget for GA is set at 50 due to scalability concerns, as evaluating more solutions obtained by applying different sequences of transformations to the malware app would significantly increase time overheads. Moreover, our preliminary experiment suggests considering 9 as the population size of the GA-based method. Note that a large population size negatively affects the performance of GA, as the query budget is quickly consumed by individuals in the initial generations.

Table 7 presents the results of the baseline when DREBIN is the target malware detector. As shown in Table 7, using RS in EvadeDroid outperforms GA. Specifically, RS not only leads to a 36.5% enhancement in ER but also accelerates EvadeDroid by $\approx 3\times$. These improvements are achieved with only 3 queries compared to GA's 22 queries.

5.7. Discussion

Real-world applicability. EvadeDroid demonstrates its ability to generate practical adversarial Android apps by considering real-world attack limitations, such as operating in ZK settings. We assume that EvadeDroid has no knowledge about the target malware classifiers and can only query them to obtain the labels of Android apps. Additionally, in some experiments, we assume that the target malware detectors only provide hard labels in response to the queries. The performance of EvadeDroid in various experiments validates its practicality. In a hard-label setting, it efficiently evades five popular commercial antivirus products with an average evasion rate of nearly 80%. Furthermore, empirical evaluations of EvadeDroid on DREBIN, Sec-SVM, ADE-MA, MaMaDroid, and Opcode-SVM result in evasion rates of 89%, 85%, 86%, 95%, and 80%, respectively. The success of our attack can be attributed to our approach of directly crafting adversarial apps in the problem space rather than perturbing features in the classifier's feature space. From a defender's perspective, EvadeDroid can be utilized in adversarial re-training to enhance the robustness of Android malware detection against realistic evasion attacks. Appendix G includes an experiment showcasing the adversarial robustness that can be achieved with the involvement of EvadeDroid.

Functionality preserving. We extended the tool presented in Pierazzi et al. (2020), in particular the organ-harvesting component, to manipulate malware apps. This tool ensures the preservation of functionality by adding dead codes to malware apps without affecting their semantics. Specifically, it incorporates opaque predicates, an obfuscated condition, to inject adversarial payloads into the apps while remaining unresolved during analysis, ensuring the payloads are never executed. Generally, verifying the semantic equivalence of two programs (e.g., a malware app and its adversarial version) is not trivial (Barr et al., 2015). Therefore, similar to the prior studies (Li and Li, 2020; Pierazzi et al., 2020; Yang et al., 2017), our primary goal is to consider the installability and executability of apps to verify the correct functioning of the adversarial apps. To this end, we developed a scalable test framework that installs and executes adversarial apps on an Android Virtual Device (AVD) and conducts monkey testing (Android Studio, 2023) to simulate random user interactions with the apps to guarantee the stability of the apps. Furthermore, taking inspiration from prior research (Li et al., 2023), we incorporate a log statement within the opaque predicate to ensure that the functionality of the manipulated apps remains unchanged. By monitoring the absence of log outputs, we can ascertain that the injected payloads are not executed. We select 50 adversarial apps, representing diverse malware families, for which their original malware apps can be installed and executed on the AVD without any issues. These apps are then subjected to our test framework. While the flaws in the Soot (Vallée-Rai et al., 1999) framework (e.g., the injection of payloads through Soot might result in incorrect updates to the function address table of the app), utilized in the manipulation tool (Pierazzi et al., 2020), affect the executability of a few cases, the majority of the apps passed the test.

Query efficiency. According to the experimental results obtained by applying EvadeDroid on academic and commercial malware detectors, we demonstrated that it can successfully carry out a query-efficient black-box attack. For instance, our proposed attack often only needs an average of 4 queries to generate the AEs that can successfully bypass DREBIN, Sec-SVM, ADE-MA, MaMaDroid, and Opcode-SVM. Moreover, we showed that EvadeDroid can effectively fool commercial antivirus products with less than two queries. One of the main reasons for being a query-efficient attack is due to the well-crafted transformations gathered in the action set. To maintain EvadeDroid's performance, it is crucial to periodically update the action set by incorporating newly published apps as new potential donors. Besides the quality of the action set, the presented optimization method is another important aspect of our proposed attack that can facilitate the identification of an optimal sequence of transformations, especially when the target detectors are robust to AEs (e.g., Sec-SVM). In fact, the proposed RS technique is an efficient sampling-driven search strategy that can quickly converge to a proper solution. Table 8 shows that Android evasion attacks often employ gradient-driven (e.g., gradient descent) and sampling-driven (e.g., GA) methodologies, where the latter is more practical for black-box evasion attacks because they can overcome the challenges inherent in using gradient-driven attacks in ZK settings. Specifically, gradient-driven attacks require access to precise details of target malware detectors and are limited to differentiable-based classifiers, which are not applicable to attacks operating in ZK settings. Moreover, gradient-driven techniques are not well-suited for continuous features, whereas the malware domain predominantly involves discrete features. Ultimately, gradient-masking (Papernot et al., 2017) defenses implemented in target malware detectors demonstrate effectiveness in preventing gradient-driven attacks. It is important to note that our proposed sampling-driven method demonstrates greater efficiency compared to query-based methods used in other studies (e.g., Croce et al., 2022; He et al., 2023; Xu et al., 2021, 2023). For instance, as shown in §5.3, EvadeDroid can effectively evade DREBIN with only 3 queries, whereas GenDroid needs 93 queries. Additionally, as illustrated in §5.6, our proposed RS-based strategy requires ≈ 210 seconds to bypass DREBIN, while the GA-based methods extend the evasion time to ≈ 631 seconds.

Scalability and effectiveness. Our empirical evaluations demonstrate the ability of the EvadeDroid to adapt and work effectively across a large scale of targets. Especially the results in §5.2 highlight the effectiveness of our evasion attack in bypassing diverse malware detectors

Table 8
The prevalent search strategies employed in Android evasion attacks.

Search Strategy	Description	Study
Gradient-driven	Utilizes gradients to iteratively adjust perturbations towards optimal adversarial perturbations.	Chen et al. (2018, 2019); Demontis et al. (2017); Grosse et al. (2017); Li and Li (2020); Li et al. (2021b); Pierazzi et al. (2020); Zhang et al. (2021)
Sampling-driven	Involves exploring the solution space by sampling candidate perturbations to find optimal adversarial perturbations.	Croce et al. (2022); He et al. (2023); Li et al. (2023); Liu et al. (2019); Rathore et al. (2021); Xu et al. (2021, 2023); Yang et al. (2017)

(i.e., linear vs. non-linear malware classifiers, and gradient-based vs. non-gradient-based malware classifiers) that utilize different features (i.e., syntax, opcode, and semantic features) with different feature types (i.e., discrete and continuous features). Furthermore, although manipulating applications within the problem space is inherently a time-consuming endeavor, the efficiency in querying allows our attack to autonomously generate AEs at a good speed, eliminating the need for manual and labor-intensive methods. Our empirical assessment in §5.5 also demonstrates that AEs generated by EvadeDroid to target a specific detector exhibit reusability across various malware detectors.

Potential applications. EvadeDroid shows promise for various real-world applications within the realm of Android malware detection. Security professionals and organizations involved in the development and deployment of malware detectors can utilize EvadeDroid for security testing and evaluation. For instance, they can simulate adversarial scenarios to identify vulnerabilities and enhance the robustness of their systems against real-world threats. The adversarial training capabilities of the system render EvadeDroid a helpful asset for developers seeking to strengthen malware detectors against real-world evasion attacks. Moreover, our attack can be instrumental in the development of countermeasures, allowing cybersecurity experts to understand and address potential weaknesses in existing malware detection systems.

6. Limitations and future work

In this section, we elaborate on the limitations of our proposed method, which can be considered as future work. One of the concerns of EvadeDroid is the adversarial payload size (i.e., the relative increase in the size of AEs) that might be relatively high, especially for the small Android malware apps. This deficiency may cause malware detectors to be suspicious of the AEs, particularly for popular Android applications. Improving the organ harvesting used in the program slicing technique, in particular, finding the smallest vein for a specific organ, can address this limitation as each organ has usually multiple veins of different sizes.

Additionally, EvadeDroid particularly crafts malware apps to mislead the malware detectors that use *static* features for classification. We do not anticipate our proposed evasion attack to successfully deceive ML-based malware detectors that work with behavioral features specified by dynamic analysis as the perturbations are injected into malicious apps within an `IF` statement that is always `False`. Therefore, it remains an interesting avenue for future work to evaluate how our proposed attack can bypass behavior-based malware detectors.

Furthermore, since EvadeDroid uses a well-defined optimization problem outlined in Algorithm 1, it can be extended to other platforms (e.g., Windows) if attackers offer problem-space transformations that are tailored to manipulate real-world objects (e.g., Windows Portable Executable files). This is because the transformations used in EvadeDroid can only be applied to manipulate Android applications. We leave further exploration as future work since it is beyond the scope of this study.

Finally, our study comprehensively covers various malware detection systems, employing diverse classifiers on different features with various types. However, there is an opportunity to improve the validity of our findings since the evaluation is conducted in controlled laboratory settings. Future research should delve deeper into the applicability of our adversarial attack framework in real-world environments, where dynamic factors like evolving malware landscapes and deployment scenarios may impact the attack's performance.

7. Conclusions

This paper introduces EvadeDroid, a novel Android evasion attack in the problem space, designed to generate real-world adversarial Android malware capable of evading ML-based Android malware detectors in a black-box setting. Unlike previous approaches, EvadeDroid directly operates in the problem space without initially focusing on finding feature-space perturbations. Experimental results demonstrate the effectiveness of EvadeDroid in deceiving various academic and commercial malware detectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We have provided access to our code in an online repository, and the link can be found in the manuscript.

Acknowledgement

Veelasha Moonsamy was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972.

Appendix A. Problem-space constraints

To generate realizable AEs, adversarial attacks need to consider the following four problem-space constraints (Pierazzi et al., 2020):

- **Available transformations** describe the types of manipulations (e.g., adding dead codes) that an adversary can utilize to modify malware apps.
- **Preserved semantics** constraint explains that the semantics of an Android app should be maintained after applying a transformation to the app.
- **Robustness-to-preprocessing** constraint describes the requirement that non-ML methods (e.g., preprocessing operators) should not be able to undo the adversarial changes.
- **Plausibility** constraint explains adversarial apps must look realistic (i.e., naturally created) under manual inspection.

Appendix B. Donors evaluation

In this evaluation, we assess the influence of our donor selection strategy on the performance of EvadeDroid. Two action sets, denoted as Δ_1 and Δ_2, are provided, each containing 20 transformations. The

Table B.9
The performance of EvadeDroid in attacking DREBIN when it utilizes two different action sets Δ_1 and Δ_2.

Action Set	ER (%)	Avg. No. of Queries	Avg. No. of Transformations
Δ_1	66.0	3	2
Δ_2	68.0	7	3

transformations in Δ_1 and Δ_2 are chosen at random from the collection of transformations extracted from the 10 most similar apps and the 10 least similar apps to malware apps, respectively. To understand the process of finding similar apps, refer to Section 4.3.1. We then use these action sets in EvadeDroid to transform 50 randomly selected malware apps into AEs. Table B.9 presents a comparison of the impact of Δ_1 and Δ_2 on EvadeDroid's performance. As can be seen in this table, when using Δ_1, the number of queries and transformations is significantly reduced compared to Δ_2. This finding validates that leveraging benign apps that resemble malware apps as the donors of transformations can reduce the cost of generating AEs, specially in terms of the required queries.

Appendix C. Implementation details

The proposed framework illustrated in Fig. C.8 is implemented with Python 3 and Java 8. The source code[1] of the pipeline has been made publicly available to allow reproducibility. The components of EvadeDroid's pipeline are clearly depicted in Fig. C.8. This section reviews some of the components that have not been previously described in detail in the paper.

- **Component 7.** To identify API calls in donor apps, we utilize the tool provided in Spreitzenbarth (2014). This tool leverages Apktool (Apktool, 2010) to access the DEX files of Android apps, which are represented as smali files. It employs string analysis techniques to scan these files and identify the API calls present within them.
- **Component 8.** We extend the tool presented in Pierazzi et al. (2020) to extract API calls from donors because this tool, which is based on the Soot framework, originally harvests Activities and URLs only.
- **Component 10.** The tool presented in Pierazzi et al. (2020) has also been used to inject gadgets into malware apps (i.e., hosts). This tool ensures the fulfillment of both the preserved-semantic and robustness-to-preprocessing constraints by utilizing opaque predicates (Moser et al., 2007) for transplanting the gadgets into hosts. The opaque predicates employed in the tool are obfuscated condition statements that encapsulate the injected gadgets. During runtime, these statements always evaluate to `False`, thereby preserving the semantics of malware apps as the injected gadgets remain unexecuted. Furthermore, the preprocessing operators are unable to eliminate the injected gadgets as the result of the statement cannot be statically resolved from the source code during design time. It is important to note that the generated AEs are plausible. This is because the manipulation of malware apps involves the injection of realistic gadgets found in benign apps. Additionally, the injection of the gadget occurs in unnoticeable injection points, maintaining the homogeneity complexity of the host's components. The inclusion of gadgets may enhance EvadeDroid's performance by introducing more features in the manipulated apps. For further insights into the tool, we refer readers to Pierazzi et al. (2020).

[1] https://anonymous.4open.science/r/EvadeDroidMain-1E69.

Appendix D. Android malware detectors

DREBIN (Arp et al., 2014) and **Sec-SVM** (Demontis et al., 2017) are two prominent approaches in Android malware detection. DREBIN utilizes binary static features and employs linear Support Vector Machine (SVM) for classification. It extracts various features, including requested permissions and suspicious API calls, from the Manifest and DEX files of APKs through string analysis (Gibert et al., 2020). These features are then used to construct a feature space for the classifier. In DREBIN, each app is represented by a sparse feature vector, where each entry indicates the presence or absence of a specific feature. Secure SVM (Sec-SVM) is an enhanced version of DREBIN that aims to enhance the resilience of linear SVM against adversarial examples. The core concept behind Sec-SVM is to increase the cost of evading the model when generating adversarial examples. Compared to DREBIN, Sec-SVM relies on a larger set of features for malware detection, making it more challenging to evade. Since Sec-SVM is a sparse classification model, it leverages a greater number of features to improve its malware detection capabilities

ADE-MA (Li and Li, 2020) is an ensemble of deep neural networks (DNNs) that is strengthened against adversarial examples with *adversarial training*. The adversarial training method tunes the DNN models by solving a min-max optimization problem, in which the inner maximizer generates adversarial perturbations based on a mixture of attacks, i.e. iterative "max" Projected Gradient Descent (PGD) attacks.

MaMaDroid (Onwuzurike et al., 2019) utilizes static analysis to detect Android malware. The goal of MaMaDroid is to capture the semantics of an Android app by employing a Markov chain based on abstracted sequences of API calls. The process begins with generating a call graph for each Android app. From this call graph, the sequences of API calls are extracted and abstracted into different modes, including families, packages, and classes. Subsequently, MaMaDroid constructs a Markov chain for each abstracted API call in an APK, where each state represents a family, package, or class, and the transition probabilities indicate the state transitions. Finally, feature vectors incorporating continuous features are created based on the generated Markov chains.

Opcode-SVM (Jerome et al., 2014) is an Android malware detection method that utilizes static opcode-sequence features instead of predefined features. This approach focuses on performing n-gram opcode analysis to represent apps in a feature space, where a malware classifier is constructed. Specifically, the method employs a linear SVM with 5-gram binary opcode features to effectively detect Android malware.

Appendix E. Experimental settings

Android malware detectors. We built DREBIN, Sec-SVM, MaMaDroid, and ADE-MA based on their available source codes (i.e., Bitbucket, 2020; Github, 2020; S2Lab, 2020) that have been published in online repositories. Moreover, we have reproduced Opcode-SVM based on the implementation details provided in Jerome et al. (2014). The hyperparameters of the reproduced malware detectors are similar to those considered in their original studies (Jerome et al., 2014; Li and Li, 2020; Onwuzurike et al., 2019; Pierazzi et al., 2020). Note that in our paper, the reproduced MaMaDroid (Onwuzurike et al., 2019) is based on the K-Nearest Neighbors (KNN) algorithm with $k = 5$. This malware classifier operates in the family mode in all experiments. KNN algorithm is used in MaMaDroid as we empirically concluded that KNN performs better on our dataset than other classifiers employed in Onwuzurike et al. (2019).

Baseline evasion attacks. We implemented Sparse-RS, ShadowDroid, and GenDroid with Python 3 based on their relevant studies (i.e., Croce et al., 2022; Xu et al., 2023; Zhang et al., 2021). Moreover, Pi-Attack (Pierazzi et al., 2020) has been built based on their available source codes published in S2Lab (2020).

EvadeDroid. Besides query budget Q and the allowed adversarial payload size α that have been mentioned earlier, n is another hyper-

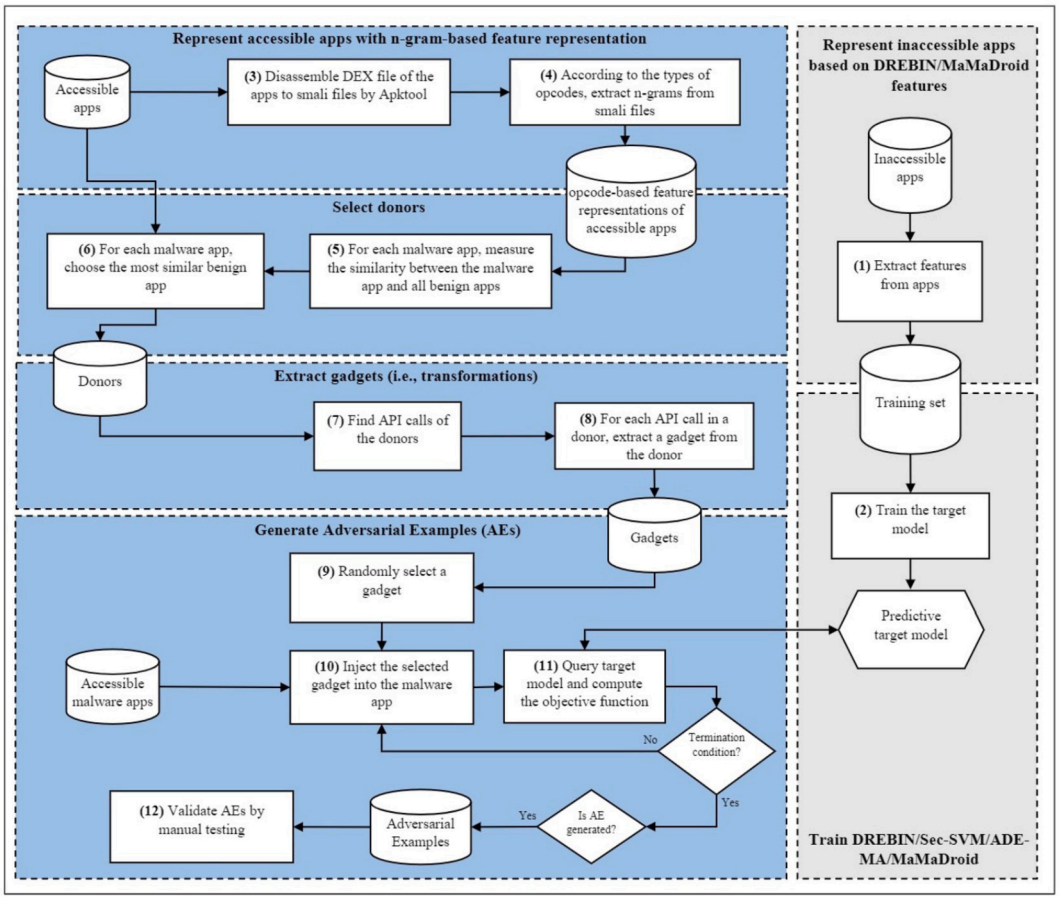

Fig. C.8. The details of the proposed framework. The blue and gray areas represent the workflows of EvadeDroid and target black-box malware detection, respectively.

parameter that shows the length of overlapping sub-string of opcodes' types in n-gram-based feature extraction. In this study, we consider $n = 5$ because in Jerome et al. (2014), the authors have shown that the best classification performance for opcode-based Android malware detection can be achieved with the 5-gram features. Furthermore, we select the top-100 benign apps as suitable donors for gadget extraction. Note that we consider 100 donors as organ harvesting from donors is a time-consuming process.

Appendix F. Baseline attacks

PiAttack (Pierazzi et al., 2020) is a white-box attack in the problem space that generates real-world AEs using transformations called gadgets. This attack comprises two main phases: the initialization phase and the attack phase. In the initialization phase, key benign features are identified, and then gadgets corresponding to the identified features are collected from benign apps. In the attack phase, a greedy search strategy is used to find optimal perturbations by selecting gadgets based on their contribution to the feature vector of the malware app. This process is repeated until the modified feature vector is classified as a benign sample. Note that PiAttack incorporates both primary features and side-effect features into malware apps. The primary features are added to bypass detection, while the side-effect features are included to meet problem-space constraints.

Sparse-RS (Croce et al., 2022) attack is a soft-label attack that gradually converts malware samples into AEs by querying the target model. Sparse-RS, which is a gray-box attack in the malware domain, finds the l_0-bounded perturbations (i.e., the maximum allowed perturbations) via random search. Note that we set initial decay factor $\alpha_{init} = 1.6$ and sparsity level $k = 180$ similar to Croce et al. (2022) and query budget $Q = 1000$.

ShadowDroid (Zhang et al., 2021) is a black-box problem-space attack that generates AEs by building a substitute classifier, which is a linear SVM. The substitute classifier is built on binary feature space compromised by permissions and API calls. This attack makes a key feature list based on the importance of features specified by the substitute classifier. The attack adds the key features to a malware app and queries the target classifier to check if the manipulated app is classified as malware. ShadowDroid continues this process until reaching the maximum query budget or generating an AEs. We set query budget $Q = 100$, following a similar setting as in Zhang et al. (2021). Note that ShadowDroid is not fully compatible with the ZK setting as it relies on the assumption that the target detectors utilize permissions and API calls for malware detection. However, since it is a query-based problem-space attack, it serves as a proper naive problem-space baseline attack for our study.

GenDroid (Xu et al., 2023) is a black-box Android evasion attack building upon GenAttack (Alzantot et al., 2019). This query-based attack utilizes GA to discover adversarial perturbations in soft-label settings. GenDroid extends GenAttack by redesigning the fitness function, adopting a new evolutionary strategy, and incorporating Gaussian Process Regression (GPR) to guide evolution. Specifically, the fitness function is defined through a logarithmic transformation, incorporating adjustable weight parameters (α and β) and a norm-bounded perturbation. The selection process prioritizes elite individuals with higher fitness scores, and the *Softmax* function is employed to convert fitness scores into probabilities. GPR is introduced to predict fitness values for individuals in the next generation. We empirically set the population size to 8 and the maximum number of generations to 50.

Table G.10
The impact of various training strategies on the utility of DREBIN.

Model	No. of AEs	TPR (%)	FPR (%)
Standard Training	N/A	80.8	1.7
Adversarial Re-training	500	78.3	1.4
	1000	74.9	0.9
	1500	68.7	0.5
	1769	32.7	0.2

Appendix G. Data augmentation

In this experiment, we evaluate the performance of EvadeDroid in enhancing the adversarial robustness of Android malware detection. To achieve this, we transform malware samples from the original training set into AEs using EvadeDroid. Subsequently, we re-train DREBIN using the modified dataset, resulting in a model that is robust to EvadeDroid. Our empirical analysis demonstrates that incorporating AEs generated by EvadeDroid in the training set of DREBIN can effectively thwart the adversarial effect of EvadeDroid. However, the number of AEs employed has an effect on the DREBIN's utility (i.e., the original performance of DREBIN). Table G.10 reveals that the addition of more AEs to the training set reduces the TPR of DREBIN. For instance, the TPR of DREBIN is reduced by 32.7% compared to the standard training when $1769K$ malware samples in the training set are transformed into AEs. It is noteworthy that out of the $2K$ malware samples in the training set, EvadeDroid is capable of generating 1769 AEs.

References

Aafer, Y., Du, W., Yin, H., 2013. DroidAPIMiner: Mining API-level features for robust malware detection in Android. In: International Conference on Security and Privacy in Communication Systems. Springer, pp. 86–103.

Abri, F., Siami-Namini, S., Khanghah, M.A., Soltani, F.M., Namin, A.S., 2019. Can machine/deep learning classifiers detect zero-day malware with high accuracy? In: 2019 IEEE International Conference on Big Data (Big Data). IEEE, pp. 3252–3259.

Ahmed, F., Hameed, H., Shafiq, M.Z., Farooq, M., 2009. Using spatio-temporal information in api calls with machine learning algorithms for malware detection. In: Proceedings of the 2nd ACM Workshop on Security and Artificial Intelligence, pp. 55–62.

Allix, K., Bissyandé, T.F., Klein, J., Le Traon, Y., 2016. AndroZoo: Collecting millions of Android apps for the research community. In: 2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR). IEEE, pp. 468–471.

Alzantot, M., Sharma, Y., Chakraborty, S., Zhang, H., Hsieh, C.-J., Srivastava, M.B., 2019. GenAttack: Practical black-box attacks with gradient-free optimization. In: Proceedings of the Genetic and Evolutionary Computation Conference, pp. 1111–1119.

Ui/application exerciser monkey kernel description, 2023. https://developer.android.com/studio/test/other-testing-tools/app-crawler?authuser=1. (Accessed 6 June 2023).

Arp, D., Spreitzenbarth, M., Hubner, M., Gascon, H., Rieck, K., Siemens, C., 2014. DREBIN: Effective and explainable detection of Android malware in your pocket. In: Proceedings of the 21st Annual Network and Distributed System Security Symposium (NDSS 2014), vol. 14, pp. 1–15.

Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Le Traon, Y., Octeau, D., McDaniel, P., 2014. FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. ACM SIGPLAN Not. 49, 259–269.

Apktool: A tool for reverse engineering Android apk files. https://ibotpeaches.github.io/Apktool/. (Accessed 27 January 2022).

AV-TEST, 2004. Av-test. https://www.av-test.org/en/antivirus/mobile-devices. (Accessed 11 September 2022).

Bai, J., Wang, J., Zou, G., 2014. A malware detection scheme based on mining format information. Sci. World J., 2014.

Barr, E.T., Harman, M., Jia, Y., Marginean, A., Petke, J., 2015. Automated software transplantation. In: Proceedings of the 2015 International Symposium on Software Testing and Analysis, pp. 257–269.

Berger, H., Hajaj, C., Dvir, A., 2020. When the guard failed the droid: A case study of Android malware. Preprint. arXiv:2003.14123.

https://bitbucket.org/gianluca_students/mamadroid_code. (Accessed 11 September 2022).

Canfora, G., De Lorenzo, A., Medvet, E., Mercaldo, F., Visaggio, C.A., 2015. Effectiveness of opcode ngrams for detection of multi family Android malware. In: 2015 10th International Conference on Availability, Reliability and Security. IEEE, pp. 333–340.

Cara, F., Scalas, M., Giacinto, G., Maiorca, D., 2020. On the feasibility of adversarial sample creation using the Android system api. Information 11, 433.

Carlini, N., Athalye, A., Papernot, N., Brendel, W., Rauber, J., Tsipras, D., Goodfellow, I., Madry, A., Kurakin, A., 2019. On evaluating adversarial robustness. Preprint. arXiv:1902.06705.

Castillo, C., Samani, R., 2021. McAfee Mobile Threat Report. McAfee Advanced Threat Research and Mobile Malware Research Team, McAfee, Technical Report. McAfee.

Ceschin, F., Botacin, M., Gomes, H.M., Oliveira, L.S., Grégio, A., 2019. Shallow security: On the creation of adversarial variants to evade machine learning-based malware detectors. In: Proceedings of the 3rd Reversing and Offensive-Oriented Trends Symposium, pp. 1–9.

Chen, K., Wang, P., Lee, Y., Wang, X., Zhang, N., Huang, H., Zou, W., Liu, P., 2015. Finding unknown malice in 10 seconds: mass vetting for new threats at the Google-play scale. In: 24th USENIX Security Symposium (USENIX Security 15), pp. 659–674.

Chen, L., Hou, S., Ye, Y., 2017. SecureDroid: Enhancing security of machine learning-based detection against adversarial Android malware attacks. In: Proceedings of the 33rd Annual Computer Security Applications Conference, pp. 362–372.

Chen, L., Hou, S., Ye, Y., Xu, S., 2018. Droideye: Fortifying security of learning-based classifier against adversarial Android malware attacks. In: 2018 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM). IEEE, pp. 782–789.

Chen, X., Li, C., Wang, D., Wen, S., Zhang, J., Nepal, S., Xiang, Y., Ren, K., 2019. Android HIV: A study of repackaging malware for evading machine-learning detection. IEEE Trans. Inf. Forensics Secur. 15, 987–1001.

Croce, F., Andriushchenko, M., Singh, N.D., Flammarion, N., Hein, M., 2022. Sparse-RS: A versatile framework for query-efficient sparse black-box adversarial attacks. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, pp. 6437–6445.

Demetrio, L., Biggio, B., Lagorio, G., Roli, F., Armando, A., 2019. Explaining vulnerabilities of deep learning to adversarial malware binaries. Preprint. arXiv:1901.03583.

Demetrio, L., Biggio, B., Lagorio, G., Roli, F., Armando, A., 2021. Functionality-preserving black-box optimization of adversarial windows malware. IEEE Trans. Inf. Forensics Secur. 16, 3469–3478.

Demontis, A., Melis, M., Biggio, B., Maiorca, D., Arp, D., Rieck, K., Corona, I., Giacinto, G., Roli, F., 2017. Yes, machine learning can be more secure! A case study on Android malware detection. IEEE Trans. Dependable Secure Comput. 16, 711–724.

Eskandari, M., Khorshidpour, Z., Hashemi, S., 2013. HDM-Analyser: a hybrid analysis approach based on data mining techniques for malware detection. J. Comput. Virol. Hacking Tech. 9, 77–93.

Firdausi, I., Erwin, A., Nugroho, A.S., et al., 2010. Analysis of machine learning techniques used in behavior-based malware detection. In: 2010 Second International Conference on Advances in Computing, Control, and Telecommunication Technologies. IEEE, pp. 201–203.

Fuyong, Z., Tiezhu, Z., 2017. Malware Detection and Classification Based on n-Grams Attribute Similarity. 2017 IEEE International Conference on Computational Science and Engineering (CSE) and IEEE International Conference on Embedded and Ubiquitous Computing (EUC), vol. 1. IEEE, pp. 793–796.

Gibert, D., Mateu, C., Planes, J., 2020. The rise of machine learning for detection and classification of malware: research developments, trends and challenges. J. Netw. Comput. Appl. 153, 102526.

https://github.com/deqangss/adv-dnn-ens-malware. (Accessed 11 September 2022).

Grosse, K., Papernot, N., Manoharan, P., Backes, M., McDaniel, P., 2017. Adversarial examples for malware detection. In: European Symposium on Research in Computer Security. Springer, pp. 62–79.

Han, K.S., Lim, J.H., Kang, B., Im, E.G., 2015. Malware analysis using visualized images and entropy graphs. Int. J. Inf. Secur. 14, 1–14.

He, P., Xia, Y., Zhang, X., Ji, S., 2023. Efficient query-based attack against ML-based Android malware detection under zero knowledge setting. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 90–104.

Islam, T., Rahman, S.S.M.M., Hasan, M.A., Rahaman, A.S.M.M., Jabiullah, M.I., 2020. Evaluation of n-gram based multi-layer approach to detect malware in Android. Proc. Comput. Sci. 171, 1074–1082.

Jain, S., Meena, Y.K., 2011. Byte level n–gram analysis for malware detection. In: International Conference on Information Processing. Springer, pp. 51–59.

Jerome, Q., Allix, K., State, R., Engel, T., 2014. Using opcode-sequences to detect malicious Android applications. In: 2014 IEEE International Conference on Communications (ICC). IEEE, pp. 914–919.

Jusoh, R., Firdaus, A., Anwar, S., Osman, M.Z., Darmawan, M.F., Ab Razak, M.F., 2021. Malware detection using static analysis in Android: A review of FeCo (features, classification, and obfuscation). PeerJ Comput. Sci. 7, e522.

Kang, B., Yerima, S.Y., Sezer, S., McLaughlin, K., 2016. N-gram opcode analysis for Android malware detection. Preprint. arXiv:1612.01445.

Khormali, A., Abusnaina, A., Chen, S., Nyang, D., Mohaisen, A., 2019. COPYCAT: Practical adversarial attacks on visualization-based malware detection. Preprint. arXiv:1909.09735.

Ko, J., Shim, H., Kim, D., Jeong, Y.-S., Cho, S.-j., Park, M., Han, S., Kim, S.B., 2013. Measuring similarity of Android applications via reversing and k-gram birthmarking. In: Proceedings of the 2013 Research in Adaptive and Convergent Systems, pp. 336–341.

Li, D., Li, Q., 2020. Adversarial deep ensemble: evasion attacks and defenses for malware detection. IEEE Trans. Inf. Forensics Secur. 15, 3886–3900.

Li, D., Li, Q., Ye, Y., Xu, S., 2020. Enhancing deep neural networks against adversarial malware examples. Preprint. arXiv:2004.07919.

Li, D., Li, Q., Ye, Y., Xu, S., 2021a. Arms race in adversarial malware detection: A survey. ACM Computing Surveys (CSUR) 55, 1–35.

Li, D., Li, Q., Ye, Y., Xu, S., 2021b. A framework for enhancing deep neural networks against adversarial malware. IEEE Trans. Netw. Sci. Eng. 8, 736–750.
Li, H., Cheng, Z., Wu, B., Yuan, L., Gao, C., Yuan, W., Luo, X., 2023. Black-box adversarial example attack towards FCG based Android malware detection under incomplete feature information. In: 32nd USENIX Security Symposium (USENIX Security).
Liu, X., Du, X., Zhang, X., Zhu, Q., Wang, H., Guizani, M., 2019. Adversarial samples on Android malware detection systems for IoT systems. Sensors 19, 974.
Ma, Z., Ge, H., Liu, Y., Zhao, M., Ma, J., 2019. A combination method for Android malware detection based on control flow graphs and machine learning algorithms. IEEE Access 7, 21235–21245.
Maiorca, D., Demontis, A., Biggio, B., Roli, F., Giacinto, G., 2020. Adversarial detection of flash malware: limitations and open issues. Comput. Secur. 96, 101901.
Mas'ud, M., Sahib, S., Abdollah, M., Selamat, S., Yusof, R., 2016. An evaluation of n-gram system call sequence in mobile malware detection. J. Eng. Appl. Sci. 11, 3122–3126.
Moser, A., Kruegel, C., Kirda, E., 2007. Limits of static analysis for malware detection. In: Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007). IEEE, pp. 421–430.
Moskovitch, R., Stopel, D., Feher, C., Nissim, N., Elovici, Y., 2008. Unknown malcode detection via text categorization and the imbalance problem. In: 2008 IEEE International Conference on Intelligence and Security Informatics. IEEE, pp. 156–161.
Muñoz-González, L., Lupu, E.C., 2019. The security of machine learning systems. In: AI in Cybersecurity. Springer, pp. 47–79.
Onwuzurike, L., Mariconti, E., Andriotis, P., Cristofaro, E.D., Ross, G., Stringhini, G., 2019. MaMaDroid: Detecting Android malware by building Markov chains of behavioral models (extended version). ACM Trans. Priv. Secur. 22, 1–34.
Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z.B., Swami, A., 2017. Practical black-box attacks against machine learning, pp. 506–519.
Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., Cavallaro, L., 2019. TESSERACT: eliminating experimental bias in malware classification across space and time. In: 28th USENIX Security Symposium (USENIX Security 19), pp. 729–746.
Pierazzi, F., Pendlebury, F., Cortellazzi, J., Cavallaro, L., 2020. Intriguing properties of adversarial ml attacks in the problem space. In: 2020 IEEE Symposium on Security and Privacy (SP). IEEE, pp. 1332–1349.
Raff, E., Nicholas, C., 2017. An alternative to ncd for large sequences, Lempel-Ziv Jaccard distance. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1007–1015.
Rastrigin, L., 1963. The convergence of the random search method in the extremal control of a many parameter system. Autom. Remote Control 24, 1337–1342.
Rathore, H., Sahay, S.K., Nikam, P., Sewak, M., 2021. Robust Android malware detection system against adversarial attacks using q-learning. Inf. Syst. Front. 23, 867–882.
Rosenberg, I., Shabtai, A., Elovici, Y., Rokach, L., 2020. Query-efficient black-box attack against sequence-based malware classifiers. In: Annual Computer Security Applications Conference, pp. 611–626.
Salem, A., 2021. Towards accurate labeling of Android apps for reliable malware detection. In: Proceedings of the Eleventh ACM Conference on Data and Application Security and Privacy, pp. 269–280.
Sami, A., Yadegari, B., Rahimi, H., Peiravian, N., Hashemi, S., Hamze, A., 2010. Malware detection based on mining API calls. In: Proceedings of the 2010 ACM Symposium on Applied Computing, pp. 1020–1025.
Santos, I., Brezo, F., Ugarte-Pedrero, X., Bringas, P.G., 2013. Opcode sequences as representation of executables for data-mining-based unknown malware detection. Inf. Sci. 231, 64–82.
Shabtai, A., Moskovitch, R., Feher, C., Dolev, S., Elovici, Y., 2012. Detecting unknown malicious code by applying classification techniques on opcode patterns. Secur. Inf. 1, 1–22.
Sharif, M., Lucas, K., Bauer, L., Reiter, M.K., Shintre, S., 2019. Optimization-guided binary diversification to mislead neural networks for malware detection. Preprint. arXiv: 1912.09064.
https://s2lab.cs.ucl.ac.uk/projects/intriguing. (Accessed 11 September 2022).
Song, W., Li, X., Afroz, S., Garg, D., Kuznetsov, D., Yin, H., 2022. Automatic generation of adversarial examples for interpreting malware classifiers, pp. 990–1003.
Spreitzenbarth, M., 2014. Drebin feature extractor. https://www.dropbox.com/s/ztthwf6ub4mxxc9/feature-extractor.tar.gz. (Accessed 11 September 2022).
Vallée-Rai, R., Co, P., Gagnon, E., Hendren, L., Lam, P., Sundaresan, V., 1999. Soot - a Java bytecode optimization framework. In: Proceedings of the 1999 Conference of the Centre for Advanced Studies on Collaborative Research, (CASCON 1999). IBM Press, pp. 1–11.
Varsha, M., Vinod, P., Dhanya, K., 2017. Identification of malicious Android app using manifest and opcode features. J. Comput. Virol. Hacking Tech. 13, 125–138.
Venkatraman, S., Alazab, M., Vinayakumar, R., 2019. A hybrid deep learning image-based analysis for effective malware detection. J. Inf. Secur. Appl. 47, 377–389.
VirusTotal, 2004. Virustotal. https://www.virustotal.com. (Accessed 11 September 2022).
Weiser, M., 1984. Program slicing. IEEE Trans. Softw. Eng., 352–357.
Xu, G., Xin, G., Jiao, L., Liu, J., Liu, S., Feng, M., Zheng, X., 2021. OFEI: A semi-black-box Android adversarial sample attack framework against dlaas. Preprint. arXiv:2105.11593.
Xu, G., Shao, H., Cui, J., Bai, H., Li, J., Bai, G., Liu, S., Meng, W., Zheng, X., 2023. GenDroid: A query-efficient black-box Android adversarial attack framework. Comput. Secur., 103359.
Yang, W., Kong, D., Xie, T., Gunter, C.A., 2017. Malware detection in adversarial settings: exploiting feature evolutions and confusions in Android apps. In: Proceedings of the 33rd Annual Computer Security Applications Conference, pp. 288–302.
Zhang, J., Zhang, C., Liu, X., Wang, Y., Diao, W., Guo, S., 2021. ShadowDroid: Practical black-box attack against ml-based Android malware detection. In: 2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS). IEEE, pp. 629–636.